Learning to Pick: Robotic Strawberry Harvesting

An Imitation-Learning Pick-and-Place System on the LeRobot SO-101

Major Project Report

Akshit Harsola
Student ID: 25256470
Supervisor: Dr. Brian Deegan
EE5111 Intelligent Robotics Project, School of Engineering
University of Galway

***Abstract*—Strawberry harvesting is seasonal, labour-intensive, and increasingly hard to staff, but the fine visuomotor control it requires has historically resisted automation. This report documents an autonomous pick-and-place system built for the arm-level sub-problem within that task: given camera images of a scene, decide where and how to grasp a target object and complete the pick.**

The project was originally proposed at a much larger scope – a tracked mobile platform, GNSS-RTK navigation, LiDAR SLAM, and a 6-DOF manipulator – before a manufacturing defect in the first arm platform and single-semester time constraints led to a supervisor-approved descope to the arm-level task alone, delivered on the University of Galway robotics lab's LeRobot SO-101 leader-follower system.

The delivered system uses imitation learning: a human teleoperates the robot to demonstrate the task, and a policy learns to reproduce that behaviour without an explicit camera-to-robot geometric model. Three architectures were trained and compared on identical data: ACT, Diffusion Policy, and SmolVLA. Across four staged data-collection rounds the task grew from cube pick-and-place to strawberry-specific grasping, and the dataset grew with it, from 42 to 84 episodes. SmolVLA won on real-robot performance, with an 80% success rate (8 of 10 trials, 95% Wilson CI approximately 49–94%); both failures fell in the known bottom-tip slip zone.

Much of the project's engineering effort went into diagnosing real-robot failures rather than training models. An early round of poor results, for instance, was traced to a physically shifted wrist-camera mount, caught only because two independently trained policies failed in the same way. Despite the descope, the original proposal's own $\geq$80% pick-success target was met.

***Index Terms*—imitation learning, behaviour cloning, action chunking transformer, diffusion policy, vision-language-action models, SmolVLA, LeRobot, SO-101, robotic manipulation, pick-and-place, Wilson score interval**

1. Introduction

1.1 Motivation

Strawberry harvesting is labour-intensive, seasonal, and increasingly difficult to staff, which makes it a strong candidate for automation [1]. But the task is easier to describe than to automate. A picker locates ripe fruit among leaves and stems, approaches without damaging the plant, grasps with enough force to lift the fruit but not enough to crush it, and places it into a collection container – hundreds of times an hour. Commercial harvesting automation is still limited, largely because the underlying sub-problems (fine-grained object localisation from vision alone, and delicate, adaptive grasping) remain open engineering challenges; neither is close to solved.

This project addresses the arm-level sub-problem directly: a robot arm that observes a scene through cameras, decides where and how to grasp a target object, and completes the pick. That is narrower than a full field-deployable harvesting robot by design. Row-level navigation, crop-wide fruit detection and ripeness assessment, and the mechanical design questions around picking without damaging the plant all fall outside its scope; Chapter 3 explains why, and how the project arrived there. The technical contribution reported here is a controlled comparison of three learning-based approaches to the pick-and-place problem, evaluated not in simulation or on held-out data but on the physical robot, together with an account of what worked, what failed, and why.

1.2 Problem Statement

Three properties make the pick-and-place problem non-trivial. Each shaped a design decision documented later in this report.

Precision from vision alone. The arm must locate a target object and its own gripper accurately enough to complete a grasp, using only camera images as input. Converting what a camera sees into a physically accurate action is the central difficulty in any vision-guided manipulation system – and the specific problem that forced this project's central pivot, described in Chapter 3.

Closed-loop control under real-world noise. A plan computed once and executed open-loop fails as soon as the scene, the camera, or the arm's own position drifts from what was assumed at planning time. Servo motors slip, cameras shift on their mounts, lighting changes. On physical hardware, as opposed to simulation, a pipeline has to keep re-observing and correcting instead of committing blindly to its first plan.

Generalisation from limited demonstration data. Whether the underlying method is a hand-built geometric model or a learned policy, it must work beyond the exact conditions it was built or trained under. A model tuned to one camera position, one lighting setup, and one object is of little use for a task repeated under conditions that vary.

1.3 Project Objectives

The objectives of the project, as delivered, were to:

1. Build a working teleoperation-and-imitation-learning pipeline end to end, from raw demonstration recording through to a deployed, evaluable policy on the physical robot.
2. Train and compare multiple policy architectures on identical real-robot demonstration data, so that any observed performance difference could be attributed to the policy itself rather than to differences in training data.
3. Extend a general pick-and-place capability, first proven on a simple cube task, to strawberry-specific grasping.
4. Validate performance on the physical robot rather than in simulation alone, since simulation-only results do not capture the real sources of failure this report documents: camera-mount drift, gripper-release faults, and grasp-point sensitivity.

1.4 Report Structure

The rest of the report proceeds as follows. Chapter 2 covers the background this project builds on: imitation learning, the three policy architectures compared, and the statistical method used to interpret a small-sample success rate. Because understanding why the project changed shape matters for the design decisions that follow, Chapter 3 documents its full scope history, from the original, substantially broader proposal through the descope decision and the abandoned first hardware platform. Hardware and software architecture for the delivered system are described in Chapters 4 and 5.

Chapter 6 makes the report's central design-rationale argument – why imitation learning was chosen over a classical inverse-kinematics approach, and why the camera configuration was chosen as it was – before Chapter 7 examines the three trained policy architectures in detail. Data collection, training results, and real-robot evaluation are reported in turn across Chapters 8 through 10, and Chapter 11 describes the deployed control loop. Chapter 12 works through the concrete engineering problems encountered during the project and how each was diagnosed and resolved, and Chapter 13 addresses standards, ethics, health and safety, and sustainability. The system's limitations, including a documented negative result from an attempted robustness test, are discussed in Chapter 14 alongside concrete future work; Chapter 15 concludes. The appendices hold a reproducibility reference, a rehearsed question-and-answer set prepared for the project's bench demonstration, and a full source-traceability map for every factual claim in this report.

2. Background and Related Work

This chapter covers the concepts and prior work that the delivered system builds on. It comes before the project's own methodology (Chapters 4-6) on purpose: the design decisions in later chapters read against an explanation of the alternatives, not as unexplained choices.

2.1 Classical Approaches to Vision-Guided Manipulation

The traditional approach to a vision-guided pick-and-place task decomposes the problem into a pipeline of explicit, individually-calibrated stages: object detection in the image, a calibrated transform from image (pixel) coordinates to the robot's own physical coordinate frame, an inverse-kinematics (IK) solver that converts a target position into joint angles, and a low-level controller that drives the arm's servos to those angles. Each stage is a separate, testable component with its own accuracy characteristics, and the overall system's accuracy is bounded by the weakest stage in the chain.

In practice, the pixel-to-world coordinate transform is often the most fragile stage. It must be calibrated for a specific camera position and lens, and it silently degrades whenever that camera moves, the lighting changes enough to affect detection, or the calibration itself contained measurement error. Because the transform is a single hand-built (or learned but narrowly-scoped) mapping, there is no mechanism for the system to notice that its assumptions have been violated: it simply produces confidently wrong coordinates. This project's own first hardware phase, described in Chapter 3, encountered exactly this failure mode, and it is the direct motivation for the design pivot documented in Chapter 6.

2.2 Imitation Learning

Imitation learning is an alternative approach to obtaining a control policy: instead of hand-designing each stage of a pipeline, a model is trained to reproduce a behaviour directly from examples of that behaviour being performed by a human demonstrator. The simplest form, **behaviour cloning**, treats this as supervised learning: given a dataset of (observation, action) pairs recorded during human demonstrations, a model is trained to predict the action from the observation.

The appeal of this approach for a vision-guided manipulation task is that it removes the need for an explicit, separately-calibrated geometric model connecting camera coordinates to robot coordinates. The mapping from "what the camera sees" to "what the arm should do" is learned as a single function, end to end, directly from demonstrations that were themselves collected under the real camera and robot geometry, rather than being decomposed into stages that can each drift independently. The trade-off is that this mapping is now implicit rather than explicit: a trained policy provides no analytic guarantee of accuracy, and diagnosing why it fails on a particular case requires empirical investigation, comparing behaviour across conditions, inspecting training data, running controlled ablations, not tracing an error through a chain of testable equations. Chapter 6 returns to this trade-off directly, grounded in this project's own experience of both approaches.

A **policy**, in this context, is the trained model itself: it maps an observation (here, one or more camera images, and optionally the robot's own proprioceptive state and a natural-language task description) to an action, typically a change in joint angles or end-effector position. This project trained and compared three different policy architectures, summarised below and covered in full technical detail in Chapter 7.

2.3 Action Chunking with Transformers (ACT)

Zhao et al. [2] designed ACT to fix a specific failure mode of naive behaviour cloning: compounding error, where small prediction errors accumulate over a long sequence of single-step actions until the policy drifts into states it never saw during training. The core idea is **action chunking**. Rather than predicting one action at a time, the model predicts a short sequence of future actions in a single forward pass, which sharply reduces the number of independent decision points over the course of a task. ACT trains from scratch on the target task's own demonstration data, with no separate pretraining phase. Section 7.1 covers its architecture and this project's results with it.

2.4 Diffusion Policy

Chi et al. [3] took a different route with Diffusion Policy: it generates actions with the same mechanism that underlies modern image-generation models, starting from random noise and iteratively removing predicted noise until a coherent output remains, here an action sequence rather than an image. This lets the model represent multi-modal behaviour, more than one valid way of completing a motion, where a simple regression-based policy would instead average competing options into a single, often invalid, blended prediction. The cost is inference speed. Generating one action requires many sequential denoising steps, and this project measured that cost directly (Section 7.2). Diffusion Policy also trains from scratch on the target task's own data.

2.5 Vision-Language-Action Models and SmolVLA

A more recent line of work treats robot control as a downstream task of a large, pretrained multi-modal model, following the same pretrain-then-fine-tune pattern that has proven effective for large language models. A **vision-language-action (VLA)** model is pretrained on a large corpus of general robot demonstration data, potentially spanning many different robots, tasks, and environments, before being fine-tuned on a much smaller amount of task-specific data. SmolVLA [4], used as the project's final policy, is one such model, built on a vision-language backbone with a lightweight action-prediction head using flow matching, a technique related to diffusion but requiring fewer inference steps. Section 7.3 covers its full architecture, its published pretraining ablation, and this project's own fine-tuning results.

Why use a pretrained VLA model in a small-data project like this one? A policy trained entirely from scratch on a few dozen demonstration episodes has to learn visual feature extraction, task structure, and motor control all at once from that limited data. A pretrained VLA model only has to adapt existing, broadly-learned representations to the specific task, which is a much easier problem when task-specific data is scarce, as it was throughout this project (Chapter 8).

2.6 Statistical Evaluation of Small-Sample Success Rates

A recurring methodological concern in real-robot evaluation is that physical trials are expensive and slow relative to simulated or offline evaluation, so real-robot success rates are almost always estimated from small samples: in this project's case, ten trials for the final formal evaluation (Chapter 10). Reporting a raw percentage from a small sample without a confidence interval risks implying a precision the data does not support. This report uses the **Wilson score interval** [5] to quantify the uncertainty around the observed success rate. The Wilson interval is specifically designed to behave well when the sample size is small or the observed proportion is close to 0% or 100%, conditions under which the simpler normal-approximation interval is known to behave poorly. Section 10.3 details its use and the resulting interval for this project's final result.

2.7 Summary

Two broad families of approach to vision-guided manipulation have been covered here: classical geometric pipelines and learned imitation policies, along with the three specific policy architectures this project trained and compared. Chapter 3 turns to the project's own history, how it arrived at imitation learning after an initial attempt at the classical pipeline, and how its scope changed along the way.

3. Project Scope and Evolution

The system described in the remainder of this report is narrower in scope than the project originally proposed. This chapter documents that history instead of silently presenting the final, descoped system as though it were the original plan. What was attempted and abandoned, and why, matters for the design-rationale argument in Chapter 6 and for an honest reading of the project's results.

3.1 Original Proposal

The project was originally proposed, on 30 November 2025, under the title *"Autonomous Navigation between strawberry plant rows and Pick-and-Place Track-Drive Wheels Robot with Multi-Sensor Fusion and GNSS-RTK Assisted Phenotyping and Environmental Mapping."* Its scope statement, quoted here in full because the gap between it and the delivered system is substantial:

> This project involves the design, implementation, and testing of an intelligent autonomous robotic system capable of navigating strawberry crop rows, detecting ripe strawberries, and performing precision pick-and-place operations using a 6-DOF robotic arm. The robot will use a tracked mobile platform (SN1300 chassis), a RoboSoul S6 manipulator, an Astra Orbbec depth camera, an RP LiDAR A1, an onboard IMU, motor encoders, and optional GNSS-RTK corrections (SimpleRTK2B with 4G NTRIP or PointPerfect).

The original quantitative targets were ambitious and specific: at least 90% strawberry detection accuracy, at least 80% pick success rate, and row navigation accurate to within ±10 cm, all within a 16-week schedule. The associated work breakdown planned five phases: background research and

foundational technique study; feasibility testing and proof-of-concept work on SLAM, detection, and arm kinematics separately; core implementation integrating navigation, perception, and manipulation; evaluation and refinement in simulation and on tuned hardware; and finally enhancements including RTK-corrected global mapping and full field-condition robustness testing.

None of the mobile-platform, navigation, SLAM, or GNSS-RTK scope was implemented. The delivered system is the pick-and-place arm sub-problem only, on different hardware from what was originally specified. That broader scope was never attempted, for the reasons given in Section 3.4, and should not be confused later in this report with a failed attempt at it.

3.2 Phase 1: The Lynxmotion AL5D Implementation

The project's first hardware implementation, and the only part of the original plan that was substantially built, used a Lynxmotion AL5D five-degree-of-freedom robotic arm rather than the originally-specified 6-DOF RoboSoul S6 manipulator, paired with a Luxonis OAK-D Lite stereo depth camera. Table 3.1 gives the bill of materials for this phase.

The software pipeline for this phase followed the classical architecture described in Section 2.1: the OAK-D Lite's RGB and depth streams fed an HSV colour-blob detector (for the target object) and an AprilTag detector (tracking a marker fixed to the gripper, for closed-loop self-localisation), whose outputs were passed through a coordinate transform intended to convert pixel positions into real-world millimetre coordinates in the robot's frame, followed by a custom inverse-kinematics solver and a servo-level PWM controller (the SSC-32U board). The implementation was built in Python using the DepthAI v3 API, OpenCV, and the `pupil-apriltags` library, within a ROS (catkin) workspace.

By the time this phase ended, the following had been completed and verified working: full mechanical assembly and per-joint servo calibration; live RGB and depth streaming from the OAK-D Lite; a YOLOv8n object detector trained specifically for strawberry detection, achieving a mean average precision (mAP50) of 0.755 overall and 0.944 on the reliably-labelled "ripe" class, exported for both host-side and on-camera (OpenVINO blob) inference; HSV-based detection of a ping-pong-ball proxy target, used to validate the rest of the pipeline before committing to strawberry-specific vision; reliable AprilTag-based gripper self-localisation; and a working inverse-kinematics solver with a fallback strategy across multiple wrist orientations.

3.3 The Blocking Problem

The one component that was never successfully completed in this phase was the camera-to-robot coordinate transform: the step converting a detected object's pixel position into a physical target position the inverse-kinematics solver could act on. An initial attempt used a learned neural-network mapping for this transform, which proved unreliable in practice: its outputs were inconsistent enough that the full pick pipeline built on top of it failed. The documented plan at the time this phase ended was to abandon the learned approach in favour of a classical **2D homography**, calibrated from a fresh dataset of point correspondences, and then extend that 2D mapping to 3D using the OAK-D Lite's depth channel to recover object height. This plan was never executed on the AL5D hardware: the arm developed a manufacturing defect before the homography work began, and the project moved to a different platform (Section 3.4) before the coordinate-transform problem on this arm could be revisited.

This blocking problem is directly relevant to the design-rationale argument developed fully in Chapter 6: across this entire phase of the project, the single component responsible for the pipeline's failure was the hand-calibrated coordinate transform, precisely the class of component that imitation learning, adopted in the project's second phase, removes the need for.

3.4 Descope Decision

Two separate events led to the final, narrower scope of the delivered system: one was a hardware failure, the other a considered engineering choice made in response to it.

First, the AL5D arm developed a manufacturing defect that stalled hardware progress with no quick fix available, ending Phase 1 as described above before its coordinate-transform problem could be resolved.

Second, in response, the project's supervisor offered a choice between two alternative platforms: a Wlkata arm, or the university robotics lab's existing LeRobot SO-101 system. The SO-101 was chosen specifically because the lab already had it set up and working, with two cameras mounted, avoiding a hardware-sourcing delay that a replacement AL5D or a newly-provisioned Wlkata system would have introduced. This choice of platform brought with it a natural opportunity to reconsider the software approach as well: in preference to rebuilding the same classical pipeline (with its known single point of failure) on new hardware, the project adopted imitation learning, for the reasons developed fully in Chapter 6.

Together, these two decisions left the delivered project as the pick-and-place arm task in isolation (no mobile platform, no row navigation, no GNSS-RTK positioning, no LiDAR-based SLAM), implemented via a different methodology (imitation learning rather than classical inverse kinematics) than originally planned. That is a deliberate, supervisor-approved response to a hardware failure, not an incomplete attempt at the original scope. And despite the scope and platform change, the original proposal's own quantitative pick-success target, at least 80%, was achieved on the descoped task (Chapter 10).

3.5 Full Project Timeline

Figure 3.1 summarises the complete timeline from the original proposal through to final delivery, spanning both phases described above and the second phase's own internal evolution, detailed in Chapters 4 through 10.

TABLE I
PHASE 1 (AL5D) BILL OF MATERIALS

Component	Supplier	Unit price	Qty
Lynxmotion SES-V1 AL5D robotic arm kit (5-DoF, servo-driven)	Lynxmotion / RobotShop	EUR 442.79	1
OAK-D Lite depth camera (stereo depth + 12MP RGB)	Luxonis	EUR 143.57	1
SSC-32U servo controller (16-channel USB PWM)	Lynxmotion	included in kit	1
AprilTag markers (36h11 family, printed in-lab)	-	EUR 0.50	10
USB-A to USB-B cable	Generic	EUR 3.00	1
Estimated total		**EUR 594.36**	

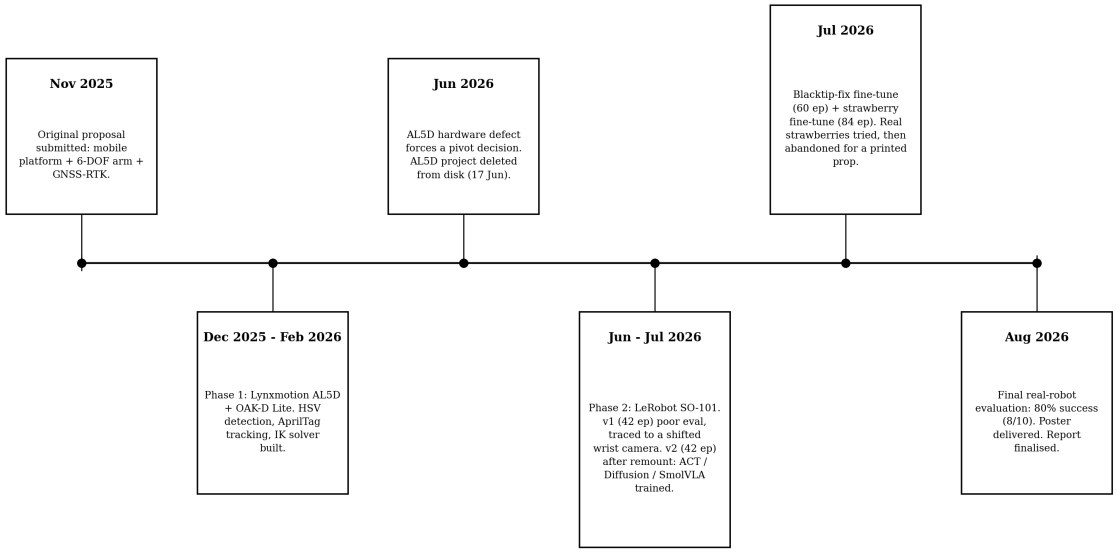

Fig. 1. Project timeline: original proposal to delivered system.

4. SYSTEM DESIGN: HARDWARE

4.1 Architecture Overview

Figure 4.1 shows the complete hardware architecture of the delivered system: two SO-101 arms (leader and follower), two RGB cameras, a training compute node, a model-hosting service, and a separate evaluation machine connected directly to the physical robot. Each component is described in the sections below.

4.2 The LeRobot SO-101 Leader-Follower Arm Pair

The core of the delivered system is the LeRobot SO-101, a low-cost, open-source robot arm platform built around two mechanically identical arms operating in a leader-follower configuration. Figure 4.2 shows the follower arm, which executes the task and carries the gripper; Figure 4.3 shows the leader arm, fitted with a handle grip for direct human teleoperation.

During data collection, a human operator moves the leader arm by hand; the follower arm mirrors the leader's joint movements via matched servo motors in real time. It is the follower's resulting motion, together with the camera views captured during that motion, that is logged as one demonstration episode (Chapter 8). This leader-follower design is what makes efficient, high-quality demonstration collection practical: the operator's own fine motor control is transferred directly to the robot's joint space, without requiring a separate teleoperation interface such as a joystick or 3D mouse.

The SO-101 was already set up in the University of Galway robotics lab, with both cameras mounted and working, at the point the project adopted it (Section 3.4). That availability, not any claimed technical superiority over the alternative Wlkata platform, is why it was chosen over sourcing a replacement for the failed AL5D.

4.3 Cameras

The system uses two plain 2D RGB webcams. An OAK-D Lite stereo/depth camera remained available from Phase 1 (Section 3.2), so this was a considered choice, made with depth sensing as a known option and set aside, not an oversight. Section 6.3 gives the full reasoning behind the trade-off, including its limitations.

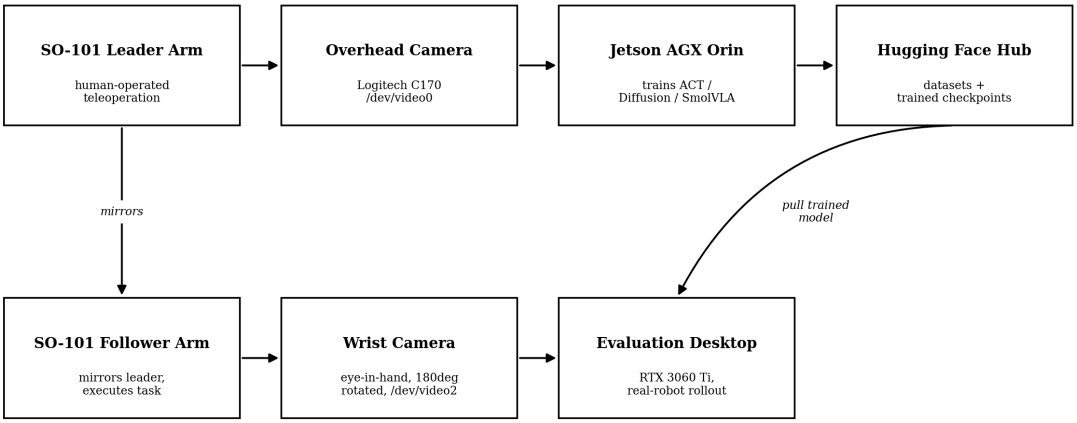

Fig. 2. Hardware and system architecture.

Fig. 3. SO-101 follower arm: executes the task and carries the gripper used for the final grasp.

- **Overhead camera**: a Logitech Webcam C170, mounted in a fixed top-down position (`/dev/video0` on the recording/evaluation machine). This camera gives global context: where the target object sits relative to the arm's

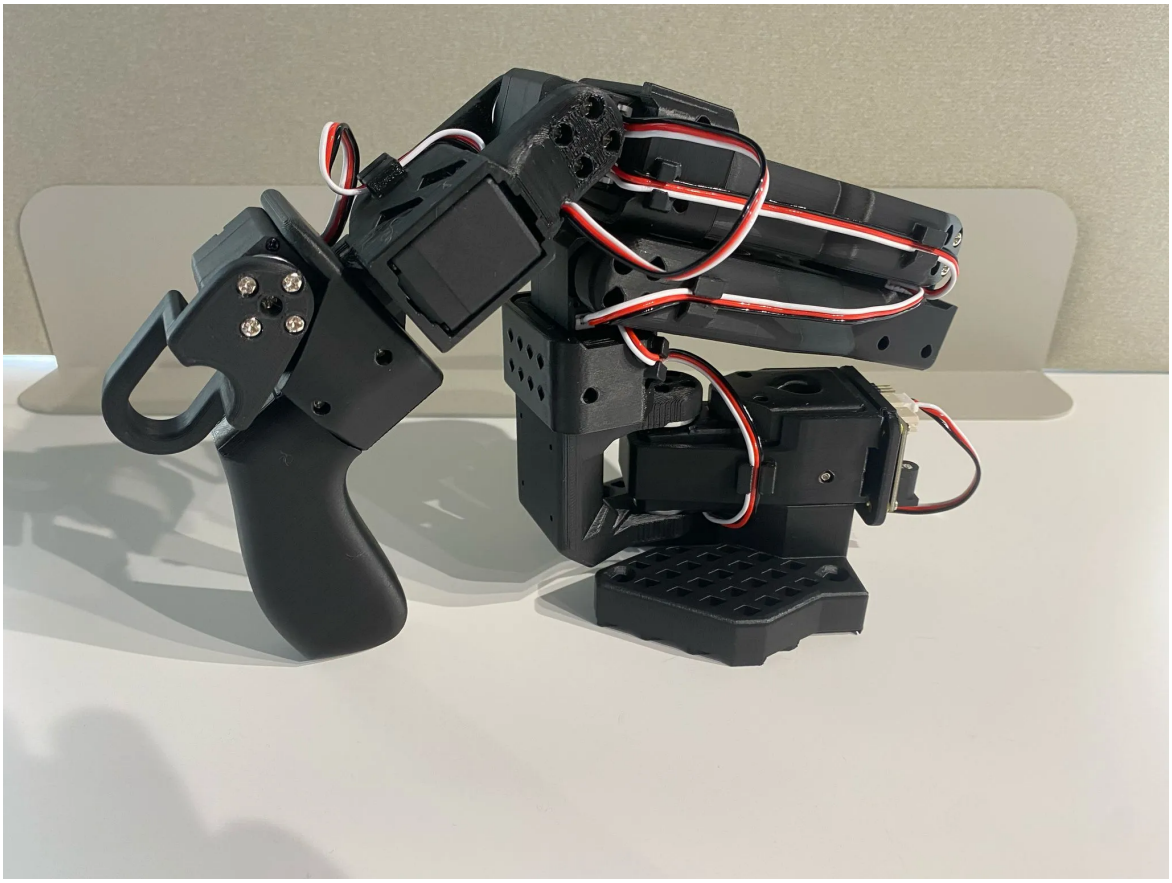

Fig. 4. SO-101 leader arm: moved by hand during teleoperation; the follower mirrors its joint movements in real time.

base.

- **Wrist camera**: a generic USB webcam mounted directly on the follower arm's gripper, rotated 180 degrees in software (`/dev/video2`). This is an eye-in-hand configuration, giving a close, largely unoccluded view of the gripper and target during the final approach and grasp, at exactly the point in the task where the overhead view becomes foreshortened and partially blocked by the gripper itself.

Both cameras record at 640x480 resolution and 30 frames per second. The project confirmed the functional importance of these two complementary roles empirically, not just by assumption: Section 9.1 describes how a physical shift in the wrist camera's mount caused every trained policy to fail identically, which shows directly what that camera contributed to the system.

4.4 Compute: Training and Evaluation Hardware

Training: Jetson AGX Orin. All policy training ran on an NVIDIA Jetson AGX Orin, an edge-AI compute platform with a CUDA-capable GPU (compute capability 8.7), running JetPack/L4T R39 with CUDA 13.2. LeRobot 0.5.2 was installed into a Python virtual environment on this machine rather than a conda environment, since the Jetson arrived without conda, pip, or LeRobot pre-installed and a lightweight venv was judged sufficient. Key pinned dependency versions were `torch==2.11.0+cu130`, `torchvision==0.26.0+cu130`, and `torchcodec==0.11.1`, chosen to satisfy LeRobot's version constraints (LeRobot 0.5.2 requires `torch<2.12.0`, while `torchcodec` in turn requires `torch>=2.11`).

Evaluation: recording desktop. Real-robot data recording and evaluation ran on a separate desktop machine (`robotics@brian-Vortex-ST-R`), fitted with an RTX 3060 Ti GPU and connected directly to the physical robot's serial ports and both cameras. Training and evaluation were deliberately kept on separate machines throughout the project: trained models were pushed to the Hugging Face Hub from the Jetson and pulled back down onto the evaluation machine for real-robot testing; checkpoint files were never transferred directly. This separation is visible in Figure 4.1 and is the origin of one of the project's recurring engineering issues, documented in Section 12.3.

4.5 Training Compute Constraint: Jetson Disk Space

The Jetson AGX Orin's on-board storage (a 59 GB eMMC module) proved to be a persistent practical constraint. Only approximately 5-8 GB remained free during active work once the operating system, JetPack, and CUDA toolkit were accounted for. A 28.7 GB USB drive was attached as overflow storage for training outputs and checkpoints; it was specifically

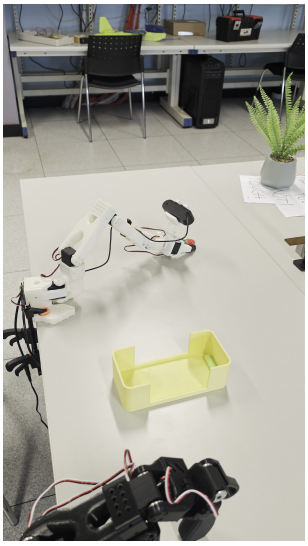

Fig. 5. Full lab setup during a recording session.

Fig. 6. Full lab setup, alternate viewing angle.

reformatted to the ext4 filesystem after both APFS and exFAT were ruled out: APFS is not usable from Linux, and exFAT lacks the symbolic-link support that the Hugging Face cache and LeRobot's checkpoint-pruning tooling both depend on. Checkpoint sizes varied considerably by policy (approximately 198 MB for ACT, 1.5 GB for SmolVLA, and 3.2 GB for Diffusion Policy per saved checkpoint), and a keep-one checkpoint-pruning script was run alongside any large-policy training run to prevent the disk from filling during a multi-hour job. This constraint caused one training run to fail outright, discussed as an engineering challenge in Section 12.2.

4.6 Physical Lab Setup

Figures 4.4 and 4.5 show the complete physical setup in the University of Galway robotics lab: the follower arm in the foreground, the yellow target bin, and the leader/teleoperation arm visible behind it.

5. System Design: Software and Methodology

5.1 The LeRobot Framework

The delivered system is built on **LeRobot**, an open-source robotics framework from Hugging Face providing teleoperation, data recording, training, and policy-deployment tooling around a standard demonstration-dataset format. LeRobot ships reference implementations of all three policy architectures compared in this project (ACT, Diffusion Policy, and SmolVLA), which is what made it practical to train and compare all three on identical data through a single, consistent pipeline, instead of reconciling three separate codebases.

The project used LeRobot version **0.5.2** throughout and deliberately did not upgrade to the later 0.6.0 release. The 0.6.0 release notes were checked against the project's needs: they did not resolve a configuration-compatibility issue the project had already identified and worked around (Section 12.3), and they introduced unspecified breaking changes that risked destabilising already-working recording and evaluation scripts for no offsetting benefit. LeRobot is a fast-moving codebase. Given this project's own experience of version drift within a single pinned release (Section 12.1), the upgrade was not worth the risk mid-project.

Two task strings were used throughout data collection, and were kept byte-identical across every recording session, since LeRobot's dataset-aggregation step keys on exact string matching to merge episodes into a combined dataset:

- Cube task: `"Pick up the blue block and place in the yellow bin"`
- Strawberry task: `"Pick up the red strawberry and place in the yellow bin"`

5.2 Demonstration Recording and Dataset Format

Each demonstration **episode** is one complete teleoperated pick-and-place attempt: the operator moves the leader arm, the follower mirrors it, and both camera feeds together with the follower's own joint positions are logged frame by frame for the episode's duration. Multiple episodes are aggregated into a single dataset and pushed to the Hugging Face Hub, from which any machine with network access, critically, the separate training and evaluation machines described in Section 4.4, can retrieve them without needing direct access to the recording hardware. Chapter 8 covers the full data-collection history across the project's four recording rounds and the reasoning behind each round.

5.3 Methodology: A Five-Stage Pipeline

The project's overall methodology, from initial hardware setup through to the final task extension, followed five sequential stages, shown in Figure 5.1.

Stage 1: Setup and calibration. The leader and follower SO-101 arms were wired, and each was calibrated using LeRobot's built-in per-joint calibration routine, which records minimum and maximum pulse widths and joint offsets for every motor. Teleoperation was verified working end to end

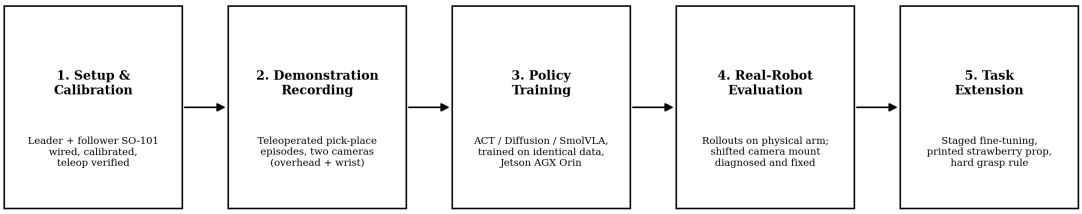

Fig. 7. Project methodology: five sequential stages.

(moving the leader by hand and confirming the follower mirrored it smoothly) before any demonstration data was recorded.

Stage 2: Demonstration recording. Teleoperated pick-and-place episodes were captured via both cameras, as described in Section 5.2. This stage was repeated across four separate rounds over the course of the project as the task and data requirements evolved (Chapter 8).

Stage 3: Policy training. All three policies (ACT, Diffusion Policy, and SmolVLA) were trained on identical aggregated demonstration data on the Jetson AGX Orin (Section 4.4). Chapters 7 and 9 cover training methodology and results for each policy.

Stage 4: Real-robot evaluation. Trained policies were deployed back onto the physical follower arm and evaluated through rollouts on real hardware, rather than in simulation. This stage is also where the project's most consequential diagnostic finding emerged: a physically shifted wrist-camera mount, discovered by comparing failure patterns across independently trained policies (Section 9.1).

Stage 5: Task extension. Once the pipeline was validated on a simple cube pick-and-place task, it was extended to strawberry-specific grasping via staged fine-tuning (Section 8.3), including an initial attempt with real strawberries that was abandoned in favour of a printed prop, and the introduction of an explicit grasp-point rule to avoid a known slip zone.

Diagnosing failures at Stage 4 turned out to be as significant a source of engineering effort as Stage 3's model training itself, a point developed fully in Chapter 12. That shapes how this report frames its own results: real-robot evaluation functions throughout as a diagnostic tool, not just a final scoring step.

6. Design Rationale

This chapter makes explicit the two design decisions most central to the delivered system, and most directly shaped by the project's own history, not by generic best practice: the choice of imitation learning over a classical inverse-kinematics pipeline, and the choice of two plain RGB cameras over the stereo depth camera that remained available from Phase 1. Both decisions were specifically raised in supervisor feedback on this project's poster draft, and both are argued here from the project's own evidence rather than from general claims about either approach's merits.

6.1 Imitation Learning versus Classical Inverse Kinematics

Section 2.1 introduced the classical vision-guided manipulation pipeline in general terms; Section 3.2 and 3.3 described this project's own attempt at that pipeline, on the Lynxmotion AL5D, and the specific point at which it failed. This section draws the two together into the project's central design argument.

The AL5D-phase pipeline used colour and AprilTag detection, a hand-calibrated pixel-to-world coordinate transform, and an inverse-kinematics solver. In practice, across the entire duration of that phase, the coordinate-transform step was the pipeline's single point of failure. It required careful calibration, and it broke under small changes in camera geometry or lighting, exactly the kind of real-world variation that a system intended for repeated, unattended use must be robust to. When the AL5D arm itself failed for an unrelated hardware reason, the project moved to the SO-101 and adopted imitation learning instead: a policy that learns the image-to-action mapping directly from demonstrations, with no explicit, separately-calibrated geometric model to maintain.

Imitation learning is not presented here as unconditionally superior; both its advantage and its cost, as this project experienced them, are worth stating explicitly.

The advantage: removing the explicit coordinate transform removes the specific failure mode already observed on the previous platform. A learned policy tolerates small scene and camera changes that would silently break a hand-coded transform, because it never relied on that transform's assumptions in the first place.

The cost is harder to live with day to day. A learned policy needs many demonstrations to generalise (this project's final policy was trained on 84 episodes, built up over four separate rounds; Chapter 8), and it provides no formal accuracy guarantee. When something does fail, such as the bottom-tip grasp slip documented in Chapter 10, the cause has to be diagnosed empirically: watching rollouts, comparing behaviour across conditions, forming and testing hypotheses, rather than tracing an error backward through a chain of testable equations. Section 9.1 has the clearest example. An entire early round of poor results looked, from the outside, exactly like a problem with the training run or the data itself, and was only correctly diagnosed as a hardware fault (a shifted camera mount) by comparing the failure pattern across two independently

trained policies. A classical pipeline's coordinate-transform stage is directly inspectable by contrast: its calibration data and output coordinates can both be checked against ground truth without running the whole system and interpreting its behaviour indirectly.

The imitation-learning approach was the right choice for this project given the circumstances under which it was adopted: a hardware failure under real time pressure, moving to a lab-ready platform that made rapid iteration on demonstration data far more practical than rebuilding and recalibrating a geometric pipeline from scratch. It is not presented here as a general claim that imitation learning is superior to classical inverse kinematics for vision-guided manipulation in every setting.

6.2 Camera Configuration: Two RGB Cameras, Not Stereo

The second design decision concerns perception hardware. The delivered system uses two plain 2D RGB webcams (Section 4.3) rather than a stereo or depth-sensing camera, despite a Luxonis OAK-D Lite stereo/depth camera remaining available from Phase 1. Depth sensing was a known option, considered and set aside, not an oversight.

The SO-101 lab rig came with two RGB webcams already mounted, wired, and working within LeRobot's camera pipeline at the point the project adopted the platform. Bringing the OAK-D Lite across from the AL5D phase instead would have required real integration work: writing or adapting a driver for LeRobot's camera interface, physically remounting the camera, recalibrating it for the new arm's geometry, and validating a depth channel through the entire recording and training pipeline, under a project deadline that had already been compressed by the AL5D hardware failure and the resulting platform pivot. The deliberate trade-off made was to use the already-working RGB cameras and spend the time that integration work would have consumed on collecting more demonstrations and running more training rounds instead.

The two cameras have distinct, complementary roles; neither is a redundant view of the same scene. The overhead camera gives global context, roughly where the target object sits relative to the arm's base, visible across the whole workspace. The wrist-mounted camera gives a close, largely unoccluded view for the final approach and grasp, precisely where the overhead view becomes foreshortened and is partly blocked by the gripper itself.

That split is a finding, not an assumption. When the wrist camera's physical mount shifted mid-project (Section 9.1), every policy trained on data recorded after that shift failed the same way: correct approach, missed grasp. That is exactly the failure pattern predicted if the close-range grasp cue the wrist camera supplied had been silently corrupted. With only the overhead camera, that same failure would be the expected default, since the wide view alone cannot resolve fine gripper-to-object alignment at contact range; with only the wrist camera, coarse localisation of the target within the wider scene would be lost. The project's own failure history demonstrates the two-camera configuration's value directly.

The trade-off does have a real cost. Both cameras are plain RGB sensors with no stereo or depth channel. The policy has no explicit sense of object distance beyond whatever it can infer from the two 2D views and its own proprioceptive state, and no way to disambiguate a target partially hidden behind another object using appearance cues alone. Section 13.2 documents an attempted clutter-robustness test that failed for exactly this reason. Section 14.1 discusses depth sensing, using the same OAK-D Lite this section explains the decision not to integrate, as concrete, low-effort future work that would directly address it.

6.3 Summary

Both decisions in this chapter follow the same underlying pattern. A component that had already demonstrated a specific failure mode on the project's first hardware platform was replaced with an approach that removed it, and a trade-off that saved development time under a compressed schedule was made deliberately, with its cost acknowledged rather than hidden. Read the remainder of this report against that framing: the technical account of the three trained policies in Chapter 7, the data collection and training process in Chapters 8-9, and the real-robot results in Chapter 10 describe a system built under real hardware and time constraints. Its design choices are defensible because their costs and justifications are both stated, not because either approach wins in the abstract.

7. Policy Architectures: Technical Deep-Dive

Chapter 2 introduced the three policy architectures compared in this project at a conceptual level. This chapter gives the full technical detail of each, together with how each was actually configured and measured on this project's own hardware, since the comparison in Chapter 10 depends on understanding how each architecture behaved in this specific deployment, not only what it is in principle.

7.1 ACT: Action Chunking with Transformers

ACT [2] is trained **from scratch** on the target task's own demonstration data; it has no separate pretraining phase, unlike SmolVLA (Section 7.3). Its architecture combines three ideas:

Action chunking. Rather than predicting one action at a time, ACT predicts a short sequence, or "chunk," of k future actions (for example, $k = 100$) in a single forward pass. This turns what would otherwise be a long sequence of individual decision points into far fewer effective ones, which sharply reduces the compounding-error problem that afflicts naive single-step behaviour cloning: with fewer independent predictions along the trajectory, there are fewer opportunities for small errors to accumulate into a state the policy has never encountered during training.

A conditional variational autoencoder (CVAE). During training, an encoder network compresses the actual demonstrated action sequence into a small latent "style" variable. This latent absorbs the natural variation between different human demonstrations of the same task. Two demonstrations of the same pick-and-place will never be pixel-identical in

their trajectories, the CVAE gives the model a principled way to represent that variation instead of forcing it to average the variation away. A transformer decoder then predicts the action chunk conditioned on the current observation and this latent variable. At inference time, the latent is simply fixed at its default value, since there is no "correct" demonstration style to condition on when the policy is acting autonomously.

Temporal ensembling. Rather than querying the policy only once every k steps (at each chunk boundary), ACT queries it at every single timestep, producing several overlapping chunk predictions for the current moment from different points in the recent past. These overlapping predictions are blended together, which produces smooth motion rather than the jerky, discontinuous motion that would result from switching abruptly at each chunk boundary.

Measured performance in this project. ACT achieved the fastest control rate of the three policies, at approximately 25 Hz, since it requires only one forward pass to produce an entire action chunk. Chapter 9 (v2 round) and Chapter 10 (final comparison) report its real-robot success-rate results.

7.2 Diffusion Policy

Diffusion Policy [3] is also trained **from scratch**, and generates actions using the same generative mechanism used by modern image-generation models.

Training. A real demonstrated action sequence is taken, Gaussian noise is added to it, and a network is trained to predict the noise that was added, conditioned on the current camera observation. This is repeated at many different noise levels during training.

Inference. Generation starts from pure random noise standing in for the action sequence, and the network's predicted noise is repeatedly subtracted (typically across dozens to over one hundred small steps) until a clean, valid action sequence remains.

Multi-modality. Because the entire action sequence is denoised together, and the trajectory it settles into depends on the random starting point, Diffusion Policy can represent distinct, valid ways of completing the same motion; it does not average them into a single, often physically invalid, blended trajectory. This is a known failure mode of simpler regression-based policies when a task has more than one reasonable solution.

Receding horizon control. The policy predicts a longer action chunk than it actually executes (for example, predicting 16 steps but executing only 8), then re-observes the scene and re-predicts. This balances the smoothness of committing to a multi-step plan against the reactivity of re-planning frequently.

Measured performance in this project. Diffusion Policy's iterative denoising process makes its inference speed highly sensitive to the number of denoising steps used, and this project measured that sensitivity directly on its own evaluation hardware (the RTX 3060 Ti desktop, Section 4.4) rather than relying on published figures. With the default DDPM sampler at 100 inference steps, the real-time control loop dropped to an unusable 0.9 Hz against a 30 Hz target. In practice, the policy was far too slow to control the robot at all under this setting. Switching to DDIM sampling improved this substantially: 10 steps achieved approximately 7.8 Hz, and 5 steps achieved approximately 12-14 Hz, the setting this project ultimately evaluated with. This is roughly half of ACT's control rate. The gap reflects an architectural trade-off between Diffusion Policy's greater representational flexibility (multi-modality) and its computational cost at inference time, not a tuning oversight: the sensitivity was investigated directly, and the fastest setting practical for real-time control was the one used for evaluation.

7.3 SmolVLA: Vision-Language-Action Model

SmolVLA [4] is architecturally distinct from the previous two: it is a **vision-language-action (VLA)** model, pretrained on a large corpus of general robot demonstration data before being fine-tuned on this project's own, much smaller, task-specific dataset. This is the same pretrain-then-fine-tune pattern used for large language models, applied here to robot control.

Backbone. SmolVLA's backbone is SmolVLM2-500M, pairing a SigLIP vision encoder with a compact (1.7-billion-parameter) language decoder. This backbone jointly processes multiple camera images, the robot's own proprioceptive state, and a natural-language description of the task being performed.

Action expert. A much smaller transformer, roughly 100 million parameters, sits on top of the backbone. It attends only up to a configurable middle layer of the backbone (roughly halving the compute cost relative to attending to the full depth), and predicts actions using **flow matching**: learning to predict a direct correction vector that pushes a noisy action guess toward the correct trajectory. This requires substantially fewer inference steps than Diffusion Policy's full iterative denoising process, while retaining some of the same underlying mathematical machinery.

Pretraining. SmolVLA was pretrained on approximately 10 million frames drawn from 487 community-contributed LeRobot datasets, prior to any of this project's own fine-tuning. This is at least an order of magnitude less pretraining data than some larger VLA systems use, which is part of why the model remains small enough to fine-tune on a single consumer-grade GPU rather than requiring large-scale training infrastructure.

Asynchronous inference. Unlike ACT and Diffusion Policy, SmolVLA in this project's deployment ran with an inference server operating in parallel with the robot's control loop, not at a single fixed rate. When the queue of already-computed actions runs low, new observations are sent off for the next prediction while the robot continues executing already-queued actions. It is not locked to one fixed control-loop rate the way the other two policies are. Results tables and figures throughout this document report it as "Async (decoupled)" rather than a single Hz figure. Section 7.4 explains the architectural reasons this is a real advantage and not merely a different way of reporting the same number.

Why SmolVLA was selected as the final policy. Hugging Face's own published ablation study of SmolVLA shows its success rate on held-out tasks rising from approximately

51.7% without community pretraining to approximately 78.3% with it. Pretraining alone accounts for most of that gap. With only 84 task-specific demonstration episodes available for the final strawberry task (Chapter 8), ACT and Diffusion Policy each had to learn visual feature extraction, task structure, and motor control entirely from that limited dataset. SmolVLA, by contrast, only had to adapt its already-broadly-learned pretrained representations to this specific task, a substantially easier learning problem given how little task-specific data was available. That reasoning is why SmolVLA was carried forward through the project's staged fine-tuning rounds (Chapter 8) and evaluated as the final system (Chapter 10). It is not a claim that SmolVLA beats ACT or Diffusion Policy in general, only that it fit this project's specific data-scarce, real-time-control setting better.

7.4 Why Asynchronous Inference Can Outperform a Fixed-Rate Loop

Comparing SmolVLA's control rate to the other two policies invites an obvious objection: having no single number to report sounds like a gap in the measurement, not an advantage. The explanation is architectural. Asynchronous inference can outperform a fixed synchronous loop, even a fast one such as ACT's approximately 25 Hz, for reasons that go beyond raw speed, worth spelling out given how directly the three policies get compared later in this report.

Eliminating inference gaps. ACT predicts a chunk of future actions (for instance, 25 steps ahead), but in a plain synchronous deployment, once the robot finishes executing that chunk it must pause completely to capture a new frame and compute the next chunk before it can continue. Asynchronous inference instead runs the policy on a separate thread or server, so that the next chunk is typically already computed and ready before the current one runs out. The robot does not have to stop and wait.

Temporal integration and smoothing. Switching from one action chunk to the next synchronously tends to produce jerky, discontinuous motion at the chunk boundary, since the two chunks were computed independently and need not agree exactly at their join. Asynchronous setups, as used by LeRobot for SmolVLA in this project, instead blend a newly-computed prediction with whatever actions are still queued from the previous prediction. This form of temporal integration means SmolVLA's motion does not have to trade off speed against smoothness the way a naive fixed-rate loop can.

Adaptive error recovery. If a synchronous policy fails to grasp an object partway through executing a chunk, it has no way to react until that entire chunk finishes playing out. It is, in effect, committed to a stale plan. Because an asynchronous setup is continuously computing new plans in the background, a failure can in principle be recognised and a recovery trajectory begun immediately, in preference to waiting for an already-wrong plan to finish executing.

For this project's own comparison: ACT's approximately 25 Hz is fast precisely because it requires only one forward pass per chunk, but it remains a synchronous, fixed-rate loop with the trade-offs described above. SmolVLA's asynchronous, decoupled execution is a different execution model, and Figure 7.1 and Chapter 10 keep that distinction explicit whenever the three policies' control rates are compared directly.

7.5 Control-Rate Comparison

Table 7.1 and Figure 7.1 summarise the three policies' architectural approach and measured real-robot control rate, drawing together the figures reported in Sections 7.1-7.3.

A methodological caveat applies throughout the remainder of this report. Whenever success rates across these three policies are compared (Chapters 9 and 10), the control-rate gap shown here should be weighed alongside the raw success count. A policy running at a substantially lower or fundamentally different control rate carries a disadvantage, or in SmolVLA's case a structural advantage, that is separate from the underlying quality of its learned behaviour. Conflating the two risks a misleading comparison.

7.6 Supporting Concepts

The following terms recur throughout the remainder of this report and are defined here for reference:

- **Imitation learning** - training a policy to reproduce a behaviour demonstrated by a human: not programmed explicitly, and not trained via a reward signal (reinforcement learning).
- **Behaviour cloning** - the simplest form of imitation learning: directly supervised learning from (observation, action) pairs recorded during demonstrations.
- **Policy** - the trained model that maps an observation (camera images, robot state, and sometimes a text instruction) to an action.
- **Control rate / Hz** - how many times per second a policy produces a new action for the robot to execute. A higher rate generally allows faster reaction to what is observed, subject to the speed-versus-representational-quality trade-off discussed in Section 7.2.
- **Fine-tuning** - continuing to train an already-trained (pre-trained) model on new, smaller, task-specific data, rather than training a new model from randomly initialised weights.
- **Checkpoint** - a saved snapshot of a model's weights at a particular point during training, from which training can be resumed or against which evaluation can be run.
- **Episode** - one complete recorded demonstration of the task, from start to finish, including all camera frames and robot states along the way.

8. Data Collection and Staged Fine-Tuning

The final training dataset was not collected in a single pass. It grew across four rounds over the course of the project, each one triggered by a specific problem found during real-robot evaluation of the previous round's models. This chapter covers that growth and the reasoning behind each round; the training outcomes are in Chapter 9, and the final real-robot evaluation is in Chapter 10.

TABLE II
POLICY COMPARISON SUMMARY

Policy	Approach	Control rate
ACT	Trained from scratch, action-chunking transformer	˜25 Hz (fixed, synchronous)
Diffusion Policy	Trained from scratch, denoising-based	˜13 Hz (fixed, synchronous, DDIM-5)
SmolVLA	Fine-tuned from a pretrained vision-language-action model	Async (decoupled - no fixed rate)

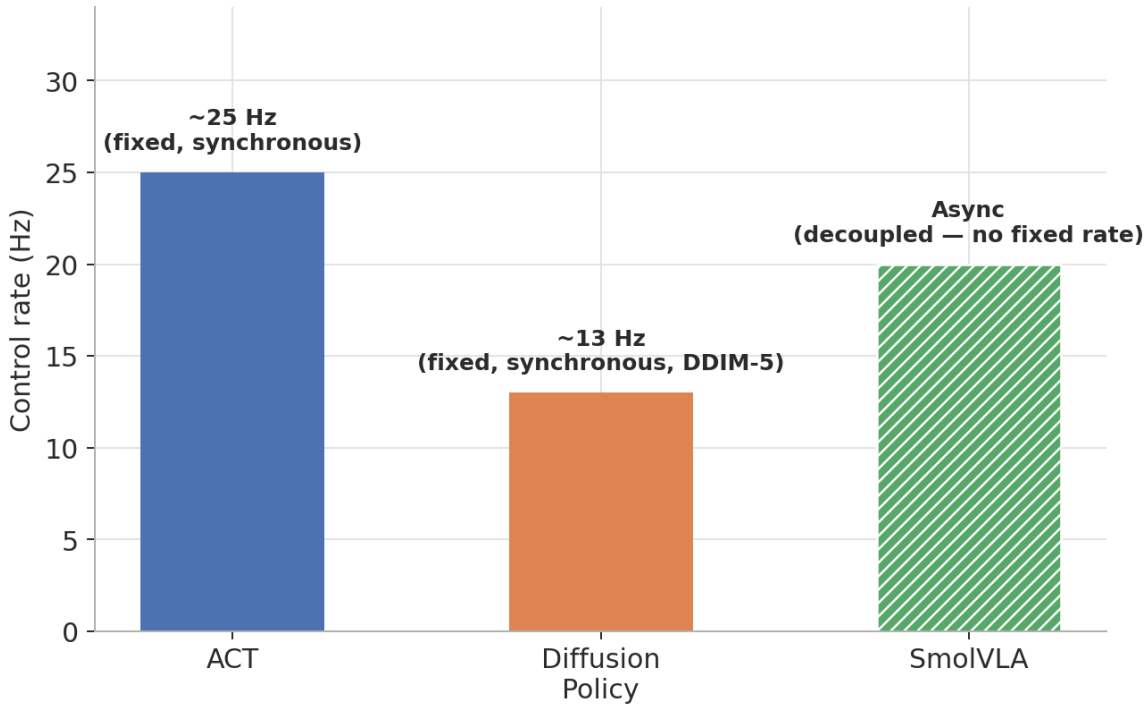

Fig. 8. Real-robot control-loop rate per policy.

8.1 Round 1 (v1): Initial Cube Recordings

The project's first demonstration dataset consisted of ten recording sessions (referred to as sessions 1 through 10), of which sessions 1-3 were later excluded due to a camera angle change partway through recording, leaving 42 usable episodes from sessions 4-10. All recordings used the cube pick-and-place task string given in Section 5.1, at 30 frames per second, using the top (`/dev/video0`) and wrist (`/dev/video2`) cameras described in Section 4.3.

As detailed fully in Section 9.1, real-robot evaluation of the policies trained on this dataset produced poor results, and the root cause was ultimately traced to a hardware fault, a physically shifted wrist-camera mount, not a data-quality problem. **This round's data and the models trained on it are not reused anywhere downstream of this point.** Because the camera geometry during recording differed from the corrected setup used for every subsequent round, mixing v1 data with later rounds would have corrupted the training set rather than simply adding more examples of the same distribution.

8.2 Round 2 (v2): Post Camera-Fix Recordings

Following the camera-mount diagnosis (Section 9.1), the wrist camera was physically remounted and its new position confirmed via a live check using LeRobot's Rerun-based visualisation tooling before any new recording began. Seven new sessions were then recorded, yielding 42 episodes and 33,353 frames, aggregated into a single combined dataset (`Akshit03/AkshitMajorProjectMIR1_v2_combined`) on the Hugging Face Hub. This is the dataset used to train and compare all three policies (ACT, Diffusion Policy, and SmolVLA) as reported in Section 9.2, and it remains the cube-task foundation that every later round built on.

8.3 Rounds 3-4: Staged SmolVLA Fine-Tuning

Real-robot evaluation of the v2-round SmolVLA model surfaced another problem: a gripper-release fault near light- or dark-coloured contact areas on the target object, where the gripper failed to open cleanly after a grasp. Two more recording rounds, each smaller than the original v2 round, addressed this fault and extended the task to strawberry-specific grasping.

Round 3 - blacktip-fix. Twelve episodes specifically targeting the gripper-release fault, plus a further six general cube-task "top-up" episodes, were recorded and aggregated together with the original v2 data into a 60-episode, 43,563-frame combined dataset (Akshit03/AkshitMajorProjectMIR1_cube_blacktip_combined). SmolVLA was fine-tuned on this combined dataset, continuing from the v2-round checkpoint instead of from the original pretrained base model: a fine-tune of a fine-tune, adding the fix on top of what the model had already learned rather than starting over.

Round 4 - strawberry task extension. Extending the task from the abstract cube target to an actual strawberry required its own data-collection effort, and this is where the project's most significant data-collection setback occurred. Real strawberries were used for two initial recording attempts and abandoned both times: **the gripper crushed or tore the fruit's skin** on contact. A proper mechanical fix (a soft-touch or compliant, rounded 3D-printed gripper fingertip) was judged infeasible within the project's remaining time budget, on the supervisor's advice. The project therefore reverted to the printed strawberry prop used earlier in the project's history (a marker-painted body with a red lower section and green cap, shown later in Figures 10.1-10.2); pursuing real fruit further was not viable within the time available.

This reversion came with a specific, deliberately enforced constraint on how the prop was grasped during recording. The strawberry prop's known **slip zone**, the bottom tip, had already caused uncontrolled slippage in an earlier, smaller batch of 18 episodes recorded before this project's staged fine-tuning process began; in that batch, the grasp point was not a controlled variable, and unstable bottom-tip grasps were simply discarded via re-recording rather than avoided by rule. For the new recording round, a **hard grasp rule** was enforced: every episode grasps the prop only at the hook or stem area, or the middle body, never the bottom tip. Three new sessions of eight episodes each (24 episodes total) were recorded under this rule.

These 24 strawberry episodes were aggregated together with the 60-episode blacktip-fix dataset from Round 3, producing the project's final combined dataset of 84 episodes and 59,865 frames (Akshit03/AkshitMajorProjectMIR1_cube_strawberry_combined), containing two distinct task strings (cube and strawberry) in a single dataset. SmolVLA was fine-tuned on this final dataset, continuing directly from the v2-round checkpoint: a parallel branch from the blacktip-fix model, not a further fine-tune of it, though the resulting dataset still includes the blacktip-fix episodes' data even though it does not inherit that round's model weights directly. This final fine-tune, smolvla_mir1_strawberry, is the policy evaluated as the project's final result in Chapter 10.

Only two episodes of real-strawberry data exist in the project's records, from the first abandoned attempt, and neither was included in any training dataset: both are contaminated by the crushing failure mode observed and represent a different visual domain (real fruit rather than the printed prop) from every other strawberry episode collected afterward.

8.4 Dataset Growth Summary

Table 8.1 and Figure 8.1 summarise the complete growth of the training dataset across all four rounds described above.

Every round after v1 used identical camera hardware and mounting, at 30 frames per second, with task strings kept byte-identical to permit clean dataset aggregation (Section 5.1). Chapter 9 covers the training outcomes each of these datasets produced.

9. Training Results

9.1 Round 1 (v1): Failure and Root-Cause Analysis

Two policies, ACT and Diffusion Policy, were trained on the original 42-episode v1 dataset (Section 8.1) and evaluated on the real robot. Both produced poor results, summarised in Table 9.1.

At face value, these results could plausibly be read as a problem with training or data quality (insufficient episodes, poor demonstration quality, or a hyperparameter issue with one or both policies). The actual root cause was none of these: the wrist-camera mount had physically shifted at some point after the original sessions 4-10 were recorded, silently corrupting the close-range visual cue the policies relied on for the final grasp.

The reasoning behind this diagnosis is worth laying out, because it is not the reasoning a training-focused investigation would have produced. Both policies, trained independently with entirely different architectures and training procedures, failed the same way. A hyperparameter problem specific to one policy's architecture would not be expected to produce an identical failure mode in an unrelated architecture trained on the same data. That both did fail identically pointed at something shared between them: not either policy's own training process, but the training data itself, and specifically the visual input it was recorded from. This is what directed the investigation toward the camera hardware instead of toward re-tuning either model, and a physical inspection of the wrist-camera mount confirmed it had shifted from its position during the original data recording.

The camera was remounted to a corrected position and this was confirmed via a live check using LeRobot's Rerun-based visualisation tool before any further recording took place (Section 8.2). v1's data and models are not reused anywhere downstream of this diagnosis: the camera geometry during v1 recording differs from every subsequent round, and mixing the two would corrupt the training set instead of augmenting it.

This episode is returned to directly in the Design Rationale (Section 6.1) as a concrete illustration of imitation learning's diagnostic cost relative to a classical pipeline: there was no analytic way to prove the camera was at fault from the training loss curves or from either model's internal behaviour alone. The diagnosis depended entirely on the empirical, cross-policy comparison described above.

9.2 Round 2 (v2): Three-Policy Training and Comparison

Following the camera fix, all three policies were trained from scratch (ACT, Diffusion Policy) or fine-tuned from their

TABLE III
DATASET GROWTH ACROSS TRAINING ROUNDS

Round	Episodes	Frames	Task(s)	Hugging Face Hub repository
v1 (sessions 1-10)	42	-	cube	discarded - see Section 8.1
v2 (camera-fix)	42	33,353	cube	AkshitMajorProjectMIR1_v2_combined
Round 3 (blacktip-fix)	60 (42 + 12 + 6)	43,563	cube	AkshitMajorProjectMIR1_cube_blacktip_combined
Round 4 (final, + strawberry)	84 (60 + 24)	59,865	cube + strawberry	AkshitMajorProjectMIR1_cube_strawberry_combined

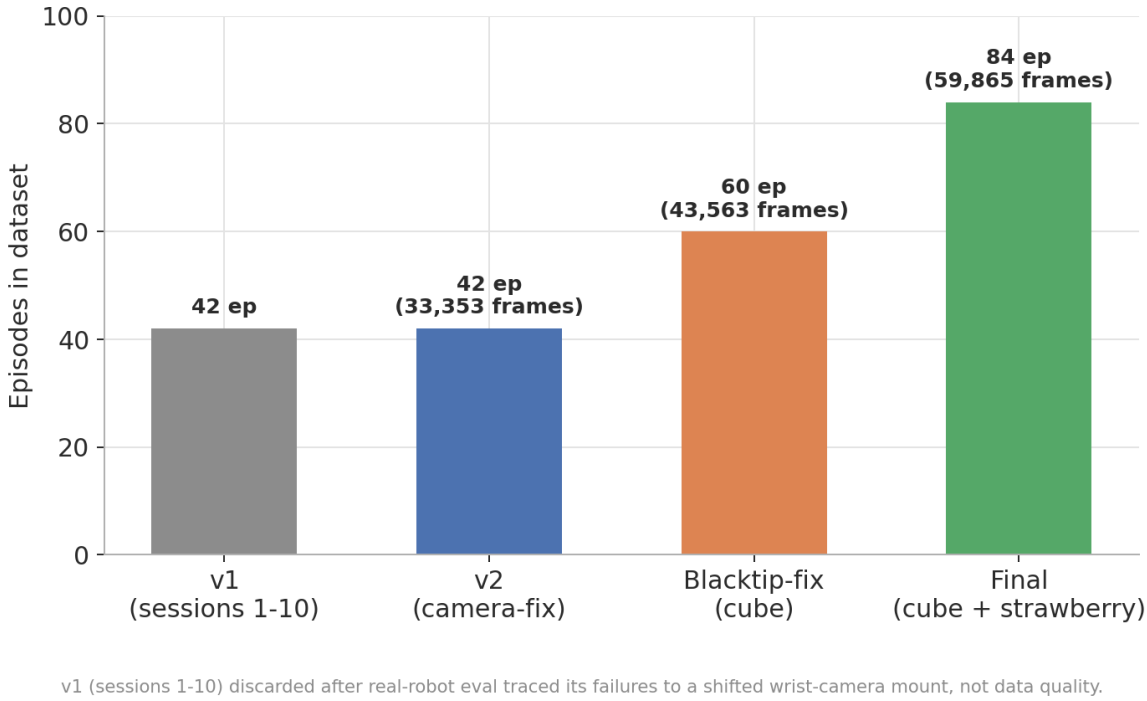

Fig. 9. Demonstration dataset growth across training rounds.

TABLE IV
V1 REAL-ROBOT EVALUATION RESULTS

Policy	Result	Observed failure pattern
ACT	1 / 10	Approach good, gripper reaches the target, grasp does not close correctly: nine consecutive failures with the identical pattern
Diffusion Policy (DDIM-5, ˜12-14 Hz)	4 / 10	Same "approach good, grasp missed" pattern, though less consistently than ACT

pretrained base (SmolVLA) on the 42-episode v2 dataset (Section 8.2), on the Jetson AGX Orin. Table 9.2 summarises the training configuration and outcome for each.

Figure 9.1 plots all three training-loss curves.

A statistical caveat applies to every training-loss comparison in this report: the three policies' loss values must not be compared to each other. ACT's reported loss is an action L1 loss; Diffusion Policy's is a noise-prediction mean-squared error; SmolVLA's is its own flow-matching-style objective. These are three different quantities on three different scales, and "Diffusion Policy's 0.007 is lower than ACT's 0.099" is not a meaningful statement about which policy learned the task better, since it compares numbers that were never measuring the same thing. The only meaningful comparison across these three policy types is real-robot success rate, reported in Chapter 10. That comparison is the reason the

TABLE V
V2 ROUND TRAINING SUMMARY

Policy	Steps	Final training loss	Wall-clock time	Hugging Face Hub repository
ACT	30,000	0.099	5h 08m	act_mir1_v2
Diffusion Policy	30,000	0.007	6h 37m	diffusion_mir1_v2
SmolVLA	20,000 (fine-tuned from lerobot/smolvla_base)	0.017	61h 36m	smolvla_mir1_v2

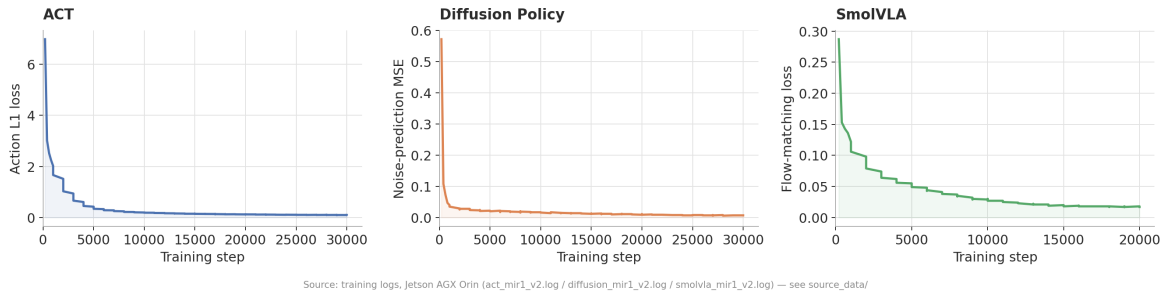

Fig. 10. Training loss curves, v2 cube dataset (42 episodes). Note: loss types differ by policy and are not directly comparable; see the caveat below.

real-robot evaluation phase of this project exists rather than relying on training metrics alone.

SmolVLA's substantially longer wall-clock training time (61 hours 36 minutes, against 5-7 hours for the other two) reflects a batch size of 64 specified by the project's supervisor, combined with the model being compute-bound on the Jetson's GPU throughout training (99% utilisation observed); no data-loading or memory bottleneck was involved.

Real-robot control-rate results for all three policies were reported in Section 7.5 (Figure 7.1) and are referenced again in the final comparison, Chapter 10. The v2-round real-robot evaluation itself was reported informally at the time as favourable for SmolVLA, but no formal, per-episode success/failure table was logged for this round: the evaluation-results document template prepared for this purpose was left blank. That gap is stated plainly here rather than filled with an invented number. What the evaluation did surface, informally but concretely, was the gripper-release fault near light- and dark-contact areas that motivated Round 3 of data collection (Section 8.3).

9.3 Rounds 3-4: Staged SmolVLA Fine-Tuning Results

Only SmolVLA was carried forward into the staged fine-tuning rounds described in Section 8.3. This was a deliberate scope decision, not an oversight. ACT and Diffusion Policy are both trained entirely from scratch on their target dataset, so extending them to a new task or fixing a narrow behavioural fault is not a small fine-tuning operation the way it is for a pretrained model. Doing so would require a full retrain on the combined dataset to avoid catastrophically forgetting what the model had already learned, and that was judged not to be a good use of the project's remaining time given SmolVLA's already-stronger real-robot performance.

Table 9.3 and Figure 9.2 summarise the three sequential SmolVLA fine-tuning rounds. Each restarts its own step counter, since each is a separate training run continuing from the previous round's checkpoint rather than a continuation of the same run.

Note that the Round 4 (strawberry) fine-tune was performed directly from the smolvla_mir1_v2 checkpoint, not from the Round 3 (smolvla_mir1_blacktip_fix) checkpoint: a parallel branch rather than a linear chain. It nonetheless includes the blacktip-fix episodes' training signal indirectly, since those episodes are baked into the combined dataset it was fine-tuned on, even though it does not inherit Round 3's model weights directly. Loss values *within* this table, since all three rows share the same SmolVLA training objective, are somewhat more meaningful to compare to one another than the cross-policy-type comparison in Section 9.2, but should still not be over-interpreted for small differences between rounds; as throughout this report, only real-robot success rate, reported next in Chapter 10, is treated as the decisive comparison.

Disk-space management for these later, longer-running fine-tuning rounds followed the approach described in Section 4.5: training output was routed to the attached USB drive, with a keep-one checkpoint pruner running alongside training, which kept the Jetson's on-board storage at a stable 6.3-7.7 GB free throughout both rounds.

10. REAL-ROBOT EVALUATION AND RESULTS

10.1 Final Evaluation Setup

The final policy, smolvla_mir1_strawberry (Round 4 of the staged fine-tuning process described in Section 8.3 and Section 9.3), was evaluated on the physical SO-101 robot performing the strawberry pick-and-place task: picking up the printed strawberry prop and placing it in the target bin. Evaluation used LeRobot's asynchronous inference workflow

TABLE VI
SMOLVLA STAGED FINE-TUNING SUMMARY

Round	Continued from	Dataset	Ep.	Steps	Wall-clock	Loss	Hub repository
v2 (baseline)	lerobot/smolvla_base	v2_combined	42	20,000	61h 36m	0.017	smolvla_mir1_v2
Round 3 (blacktip-fix)	smolvla_mir1_v2	cube_blacktip_combined	60	8,000	24.7h	0.019	smolvla_mir1_blacktip_fix
Round 4 (strawberry, final)	smolvla_mir1_v2	cube_strawberry_combined	84	10,000	30.8h	0.022	smolvla_mir1_strawberry

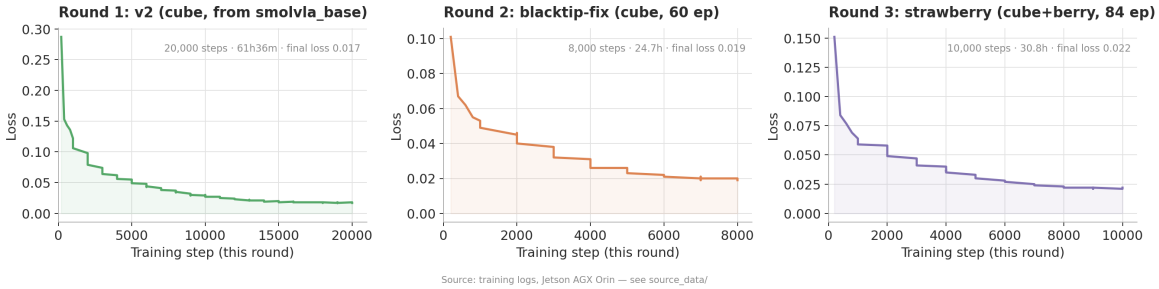

Fig. 11. SmolVLA staged fine-tuning loss across three sequential rounds.

in preference to the synchronous `lerobot-rollout` command, since the latter was confirmed unusable for SmolVLA's inference speed on the evaluation hardware in an earlier round (Section 4.4, Appendix A). Ten trials were run, each independently set up and scored by direct observation of the arm.

Success, as defined throughout this evaluation, means a clean pick: the gripper closes on the strawberry prop without crushing or dropping it, lifts it clear, and places it in the target bin.

10.2 Results

The final policy achieved 8 successes out of 10 trials, an 80% success rate. Figures 10.1 and 10.2 show the gripper closing on the strawberry prop during a successful trial.

Both observed failures shared a single, specific cause: the arm attempted to grasp the strawberry prop near its bottom tip, the known slip zone identified during the Round 4 data-collection process (Section 8.3) and explicitly excluded by the hard grasp rule enforced when recording that round's demonstrations. All other grasp attempts, at the hook/stem area or the middle body, succeeded reliably. The statistical uncertainty around this 80% figure is discussed in Section 10.3.

The detail matters more than the single percentage here. It shows the policy learned most of the grasp-point rule it was trained under: the failures cluster in one narrow, repeatable mode instead of spreading unpredictably across different causes. Section 14.1 discusses concrete, low-effort remediation for exactly this failure mode.

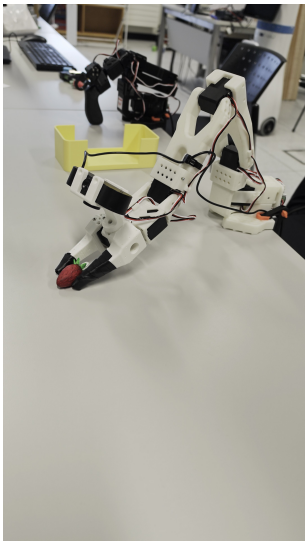

Fig. 12. Follower gripper closing on the strawberry prop.

10.3 Statistical Interpretation: the Wilson Score Interval

An 80% success rate computed from only ten trials should not be presented as a precisely known figure. This report instead follows the standard approach introduced in Section 2.6 and pairs the point estimate with a 95% Wilson score confidence interval, computed as approximately 49% to 94% for 8 successes out of 10 trials.

The Wilson interval is used here rather than the simpler normal-approximation interval because it behaves sensibly under exactly the conditions this evaluation involves: a small sample size, and an observed proportion (80%) reasonably

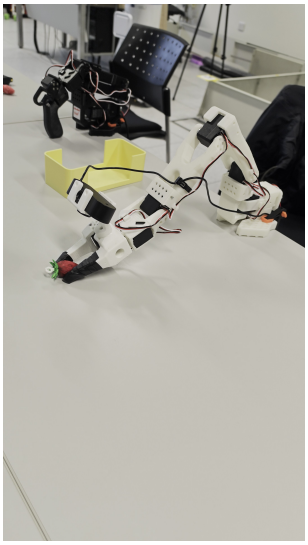

Fig. 13. Gripper-strawberry contact, alternate evaluation frame.

close to the upper bound of 100%, both conditions under which the normal-approximation interval is known to produce unreliable results. In practice, this means the true underlying success rate of this policy could plausibly sit anywhere within roughly 49% to 94% on the evidence collected. That is a wide range, and it should discipline how the 80% figure is used and interpreted elsewhere in this report and in any presentation of this project's results. **This result should be described as a pilot or indicative measurement, not a validated benchmark.** A claim intended to stand as a benchmark-grade success-rate figure would need on the order of 50 to 100 or more trials to narrow this interval to a useful width, discussed as future work in Section 14.2.

10.4 Context: The Scope of the Formal Evaluation

These ten trials do not represent the sum total of real-robot testing performed over the course of the project. They are the final, formal evaluation batch, run after the last fine-tuning round (Round 4) concluded, and the clean, controlled number this report is willing to quote as a defensible before/after comparison point.

Informal rollouts were run after essentially every training and fine-tuning round throughout the project, with the arm's behaviour watched live and any anomalous behaviour flagged for investigation. This informal testing, not the formal ten-trial batch, is how the project's two biggest diagnostic findings were caught: the wrist-camera-mount shift (Section 9.1) and the gripper-release fault near light/dark contact areas (Section 8.3). Neither appeared in any log; both were caught only by watching the physical robot operate. The formally-quoted ten-trial result therefore sits on top of a much larger amount of qualitative, real-robot testing than the number alone conveys, even though only the final batch is reported with a formal success/failure count and a statistical interval. Given more time in the lab, a substantially larger structured evaluation batch would be the natural next step, discussed in Section 14.2.

11. Control Loop and System Integration

Figure 11.1 shows the control loop of the final deployed system, as evaluated in Chapter 10. This chapter describes how the components introduced separately in earlier chapters, namely the two cameras (Section 4.3), the SmolVLA policy (Section 7.3), and the follower arm (Section 4.2), operate together as a closed system during a real-robot trial.

At each control step, both cameras capture a fresh frame. The SmolVLA policy processes these frames, together with the robot's own proprioceptive state, into a chunk of joint-angle changes, which the arm then executes toward a grasp-and-place action. What most distinguishes this from a naive vision-guided pipeline is that **the loop is closed**: the scene is re-observed and the plan re-computed at every control step, instead of the policy planning once from an initial observation and executing that plan open-loop. That is what lets the system correct for small errors as a trial proceeds, a slightly imprecise initial approach, say, or a small amount of drift in the arm's own position; it does not commit irreversibly to whatever plan was valid at the start of the trial.

Combined with the asynchronous inference architecture explained in Section 7.4, this closed-loop property is part of why SmolVLA was selected as the final policy (Section 7.3), beyond its raw success-rate advantage reported in Chapter 10.

12. Engineering Challenges Encountered

This chapter collects the concrete engineering problems encountered over the course of the project into one place, each following the same structure: what happened, how it was diagnosed, and how it was resolved. Two of these, the wrist-camera mount shift and the gripper-release fault, are described in full where they first arise, in Chapters 8 and 9; they are referenced again here for completeness. The remaining two are documented in full here.

12.1 API Version Drift

LeRobot is a fast-moving open-source codebase, and setup notes written earlier in the project no longer matched the library version actually installed on the Jetson AGX Orin by the time training began. This happened repeatedly across the project. The dataset-aggregation function moved from `lerobot.common.datasets.aggregate` to `lerobot.datasets.aggregate`. The `aggregate_datasets()` function's `push_to_hub` argument was removed, requiring a separate explicit push step afterward. The command-line interface moved from raw script invocations (`python lerobot/scripts/train.py`) to proper installed entry points (`lerobot-train`, `lerobot-record`, and others). The Hugging Face authentication command changed from `huggingface-cli login` to `hf auth login`, and the dataset metadata format changed from a `meta/tasks.jsonl` file to a `meta/tasks.parquet` file, which needed a different tool to inspect.

Resolution and lesson. No single fix addresses version drift of this kind. The practical response for the rest of the project was to treat the installed library's own documentation

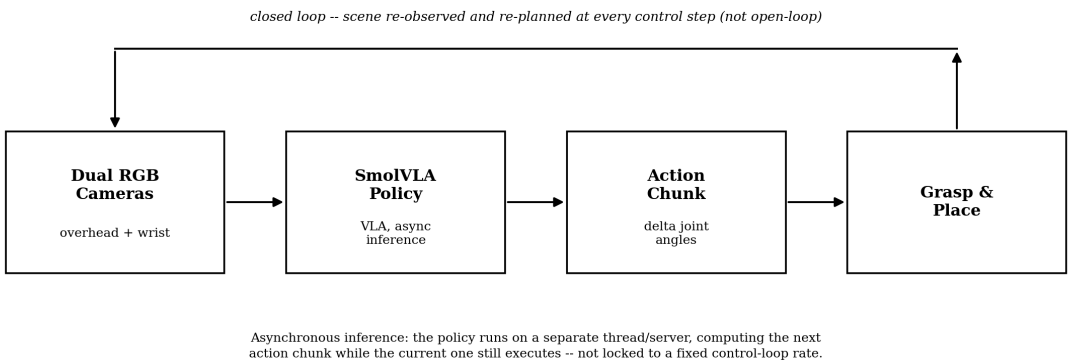

Fig. 14. Real-robot control loop, final SmolVLA deployment.

and configuration files, its `AGENT_GUIDE.md`, `AGENTS.md`, and `pyproject.toml`, as the authoritative reference for the exact version in use; setup notes written earlier, however recently, were not trusted on their own.

12.2 Jetson Disk-Space Exhaustion

As described in Section 4.5, the Jetson AGX Orin's limited on-board storage was a persistent constraint. It stopped being a background risk and became an active failure during Diffusion Policy's 30,000-step training run in the v2 round (Section 9.2): the on-board eMMC storage filled completely and the training process crashed near step 15,000.

Diagnosis. Accumulating checkpoint files consumed the available disk space faster than it was being freed, given Diffusion Policy's comparatively large per-checkpoint size (approximately 3.2 GB, as noted in Section 4.5).

Resolution. Training resumed from the last successfully saved checkpoint, at step 10,000, using LeRobot's `--resume=true` flag. That cost 5,000 steps of progress but recovered the run without restarting from scratch. For subsequent large-policy training runs, checkpoint output was routed to the attached external USB drive instead of the Jetson's own storage, and a keep-one checkpoint-pruning script ran alongside training to delete all but the most recent checkpoint on a short poll interval. It did not recur for the rest of the project.

12.3 Cross-Machine Configuration Incompatibility

Because training and evaluation deliberately ran on separate machines (Section 4.4), with trained models transferred via the Hugging Face Hub instead of direct file copying, a version-compatibility issue between the two machines' LeRobot installations surfaced as a concrete failure rather than a theoretical risk.

What happened. Attempting to load a policy trained on the Jetson directly from its Hugging Face Hub repository on the separate evaluation machine failed with a configuration-decoding error: `draccus.utils.DecodingError: The fields 'pretrained_revision' are not valid for ACTConfig` (and the equivalent error for `DiffusionConfig` and `SmolVLAConfig` policies encountered later).

Diagnosis. The LeRobot version used for training wrote a `pretrained_revision` key into the saved model's `config.json` file. The evaluation machine's installed LeRobot version used a stricter configuration decoder, and it rejected the key as unrecognised rather than silently ignoring it.

Resolution. The fix, applied identically each time the problem came up: download the affected model's files locally on the evaluation machine with `snapshot_download` instead of loading directly from the Hub repository reference, then strip the offending `pretrained_revision` field from the downloaded `config.json` with a short one-line script, and pass the local directory, not the Hub repository ID, to the policy-loading command. This is purely a version-compatibility quirk between the two machines' LeRobot installations and carries no implication about training quality. It recurred more than once across the project, whenever a newly-trained model was first loaded on the evaluation machine.

12.4 Wrist-Camera Mount Shift

Described in full in Section 9.1: an entire early round of poor real-robot results (ACT 1/10, Diffusion Policy 4/10) looked at first like a training or labelling defect, but the actual cause was a physical hardware fault: the wrist camera's mount had shifted position after the original demonstration data was recorded, silently corrupting the close-range visual cue both policies relied on for the final grasp. The diagnosis depended on comparing the identical failure pattern across two independently-trained policies (Section 9.1); the fix was to physically remount the camera and re-record all affected data (Section 8.2), with no change made to either policy's training procedure. This is the single most consequential engineering finding of the project and is discussed again in the Design Rationale (Section 6.1) and Conclusion (Chapter 15).

12.5 Gripper-Release Fault Near Light/Dark Contact Areas

Described in full in Section 8.3: informal real-robot testing of the v2-round SmolVLA model surfaced a gripper-

release fault occurring specifically near light- or dark-coloured contact areas on the target object, where the gripper failed to open cleanly after a grasp. This was resolved through the project's standard diagnose-then-targeted-recording pattern: twelve episodes specifically targeting the fault, plus six general top-up episodes, were recorded and used to fine-tune the affected model (Round 3, Section 8.3), avoiding the need for a full retrain or an architectural change.

13. Standards, Ethics, Health and Safety, and Sustainability

The project's original proposal (Section 3.1) documented these considerations for the originally-scoped mobile-platform-and-arm system. A separate paper prepared earlier in the project, co-authored with Prassidh Patel, covers relevant standards for that broader system in more depth: ISO 18497 (safety of highly automated agricultural machines), IEC 60204-1 (electrical safety of machinery), IEEE 802.11 wireless communication, GNSS-RTK correction protocols, and IEC 60825-1 laser safety for LiDAR and structured-light sensors. **Most of that discussion does not apply to the delivered system.** There is no mobile platform, no LiDAR, no GNSS receiver, and no ROS-based navigation stack in the final LeRobot-based pipeline. This chapter restates each consideration as it actually applies to the system described in this report and does not carry forward standards discussion for hardware that was never built.

13.1 Health and Safety

The delivered system involves a servo-driven robot arm pair (leader and follower SO-101 units), USB-connected cameras, and a Jetson AGX Orin compute unit. It does not involve the LiPo battery systems, high-current power electronics, or mobile chassis that the original risk register was written to address.

The residual physical risks in the delivered system are the pinch and trip hazards typical of any servo-driven arm with exposed joints and cabling, plus standard electrical-equipment handling practice for the Jetson and USB peripherals. Both arms were clamped securely to a fixed table for every recording and evaluation session. New or newly fine-tuned policies were always tested at reduced speed on their first real-robot rollout before any full-speed trial. All lab sessions ran with supervised access to the University of Galway robotics lab, with Darragh Mullins, Senior Technical Officer in the School of Engineering, providing technical support throughout.

13.2 Applicable Standards

Of the standards discussed in the original scope's engineering-standards paper, only USB-IF standards governing the camera and serial-arm USB connections remain directly applicable to the delivered system, and their role is unchanged from that earlier discussion. The delivered system does not use ROS or ROS 2 at any point in its pipeline. LeRobot has its own dataset-format and coordinate conventions instead of the ROS 2 Enhancement Proposals (REP 103, REP 105) the original paper discussed, so that discussion does not carry forward here. ISO 18497 (automated agricultural machinery safety) and IEC 60825-1 (laser product safety) likewise do not apply: there is no field-deployed autonomous machinery in scope, and both cameras used are passive RGB sensors with no laser emitter, unlike the LiDAR and structured-light depth sensors specified in the original proposal. This is stated here for scope-appropriate reporting, not left as a silent omission.

13.3 Ethics and Data Protection

No human personal data was collected at any point during this project. The camera data recorded throughout captures only the robot's own workspace, the arm itself, and the task props, the cube and the printed strawberry prop, not people, consistent with the assessment made in the original project proposal. No ethical approval was required for the work described in this report.

13.4 Sustainability

The original proposal's framing under United Nations Sustainable Development Goal 12 (responsible consumption and production) was motivated by the potential for labour-efficient automated harvesting to reduce waste and improve resource efficiency in agricultural production. That motivation remains valid for this line of work long-term, even though the delivered prototype demonstrates only the arm-level pick-and-place subproblem rather than a deployed field-harvesting system. This report makes no claim that the delivered system itself has a measurable sustainability impact; what carries forward from the original proposal is motivation for the broader research direction, not a property of the descoped deliverable.

14. Limitations and Future Work

14.1 Stated Limitations

The limitations described in this section are drawn directly from the delivered system's own documented behaviour and from a specific negative-result test attempted during the project, not from general speculation about where a system of this kind might fail.

Environmental generalisation. Both training and the final evaluation (Chapter 10) ran under consistent laboratory lighting, on an uncluttered table, with a single prop present at a time. The policy operates on RGB input only, with no depth channel and no explicit lighting normalisation. None of the demonstration data recorded across the four rounds (Chapter 8) included distractor objects, shadow conditions, or glare, so there is no evidence the policy has learned to be invariant to any of these. Degraded performance under harsher lighting or a cluttered scene is the expected default until tested and addressed, not a surprising failure.

A clutter-robustness test was attempted and did not produce valid results; that negative result is documented here, not omitted. After confirming that backup servo motors were available in case of damage during testing, a lab session (assisted by labmate Abhishek) was dedicated to testing the system with the strawberry prop partially hidden in a plastic

plant, intended to approximate foliage occlusion. The attempt did not go smoothly. The prop's own weight meant it would not hold a stable position simply resting among the plant's leaves, so it was instead wedged between branches to fix its position. That solved the positioning problem but introduced a new one: with the prop wedged among the branches, the overhead camera could not see it at all, confirmed by watching the live camera feed during the attempt. Several alternative placements were tried, seeking a position that kept the prop both plausibly "in clutter" and visible to the overhead camera, and none was found within the time available for the session.

No valid test configuration was achieved, and no clutter-condition success-rate result exists for this system. The failure of the test itself is treated here as informative, not as a gap in testing: it is a direct, physical demonstration of the limitation described above, that the system depends entirely on an unoccluded RGB view of the target and has no mechanism for reasoning about a partially hidden object. This is a specific, diagnosed cause of test failure, not an untested assumption.

The final success-rate figure (Chapter 10) is a pilot-scale result, with a 95% confidence interval spanning roughly 49% to 94%. It should not be cited as a validated benchmark figure without a substantially larger evaluation batch, as discussed in Section 14.2 below.

Both observed failures in the final evaluation shared a single cause: a grasp attempted at the strawberry's bottom tip. This is as much a positive framing as a limitation. It indicates the policy learned most of its intended grasp-point behaviour correctly, leaving one well-characterised failure case in preference to a broad or unpredictable set of failures.

14.2 Future Work

The following are concrete, incremental next steps rather than proposals for redesign, and each is grounded either in the analysis above or in an approach this project has already validated successfully elsewhere in its own history.

The residual failure is concentrated in one specific approach direction rather than spread across all grasp points, so a direct, low-effort next step is targeted re-recording: additional demonstrations specifically approaching the strawberry from the bottom-tip angle, showing the policy what a *correct* recovery or avoidance looks like in exactly the situation it currently fails in. This is the same diagnose-then-targeted-recording pattern that resolved the gripper-release fault in Round 3 of this project's own fine-tuning history (Section 8.3, Section 12.5).

A mechanical change, a compliant or soft-touch gripper fingertip, would let a slightly mis-aligned grasp still hold instead of slipping. This addresses the bottom-tip failure mode's physical mechanism directly, a poor grip surface at that specific contact point, rather than relying solely on the policy learning to avoid that region entirely.

Future data collection should also deliberately vary lighting conditions and background scene content across recorded episodes, rather than only varying the target's position as every round to date has done; this is what a meaningful test of the environmental-generalisation limitation above would require.

The Luxonis OAK-D Lite stereo/depth camera used in the project's first hardware phase (Section 3.2) remains available. It was deliberately not integrated into the delivered system for the time-budget reasons explained in Section 6.2. Adding a depth channel would move the policy beyond purely appearance-based perception, addressing the clutter/occlusion limitation demonstrated by the failed test in Section 14.1, and would remove the failure class demonstrated in Section 9.1, where a physical camera shift silently corrupted a purely visual cue with no independent depth signal available to cross-check against.

Training should also stop assuming the deployment scene will always match the consistent lab conditions used throughout this project's data collection. Incorporating augmentation or domain randomisation would expose the resulting policy to a wider range of visual conditions than any single physical recording session can practically produce.

Finally, as discussed in Section 10.3, a benchmark-grade success-rate claim would need on the order of 50 to 100 or more real-robot trials to meaningfully narrow the current 49-94% confidence interval. Given more laboratory time, this is the most direct way to strengthen the confidence with which this project's headline result can be reported; further architectural changes would not.

15. Conclusion

This project set out to build an autonomous pick-and-place system for strawberry harvesting. It delivered a narrower but thoroughly validated version of that goal: a real-robot imitation-learning system that locates a strawberry prop through two RGB cameras, grasps it, and places it in a target container, with a documented, statistically-qualified success rate and a fully traced account of every major design decision and failure encountered along the way.

The project's scope changed substantially between its original proposal and its delivered form, and this report treats that change as a subject requiring explanation, not a detail to minimise. A manufacturing defect in the project's first hardware platform, combined with the practical constraints of the project timeline, led to a considered, supervisor-approved descope from a full mobile-navigation-and-manipulation system to the pick-and-place arm sub-problem alone, implemented on a different platform via a different methodology than originally planned. Read against that history, the original proposal's own quantitative pick-success target (at least 80%) being met on the descoped task is a worthwhile result, not merely a consolation figure.

Beyond the headline success-rate figure, three findings stand out as the project's central contributions. A pretrained, fine-tuned vision-language-action model (SmolVLA) outperformed two policies trained entirely from scratch on identical, limited demonstration data, consistent with published evidence that pretraining accounts for most of such a model's advantage in exactly this low-data regime. Camera-mount stability also

proved to matter as much to real-robot success as policy architecture choice: an entire early round of poor results, initially indistinguishable from a training problem, was correctly diagnosed as a hardware fault only by comparing identical failure patterns across two independently-trained policies. That diagnostic method is documented explicitly in this report because it transfers to future work on this or similar systems, regardless of which policy architecture is used. And the project's one significant residual failure mode, a grasp attempted at the strawberry's known bottom-tip slip zone, is narrow, specific, and repeatable, not diffuse; it has a concrete, low-effort remediation path already validated by this project's own resolution of an earlier, similarly narrow gripper-release fault.

The system's most significant limitation sits not in its core pick-and-place capability but in its environmental generalisation. Training and evaluation were both conducted under consistent laboratory conditions, and an explicit attempt to test performance under partial occlusion failed to even produce a valid test configuration, itself a direct demonstration of the system's dependence on an unoccluded RGB view. This is reported plainly rather than glossed over. The future work proposed in response, depth sensing, lighting and background variation in training data, and a substantially larger evaluation batch, follows directly from that specific, tested limitation rather than from generic caution.

Taken as a whole, this project shows that a real, physically deployed imitation-learning pick-and-place system can be built, iterated, and honestly evaluated within the constraints of a single academic project, including a mid-project hardware failure and platform change, provided real-robot testing is treated throughout as a diagnostic tool for uncovering the actual causes of failure, not only as a final scoring exercise.

Acknowledgment

I would like to thank my supervisor, Dr. Brian Deegan, for his guidance throughout this project, in particular for offering a practical path forward, the LeRobot SO-101 platform, when the original hardware became unworkable partway through the project, and for detailed feedback on the poster draft that directly shaped the Design Rationale and Results sections of this report. I would also like to thank Darragh Mullins, Senior Technical Officer in the School of Engineering, for technical support in the University of Galway Intelligent Robotics lab throughout the data collection and evaluation phases of this project. Finally, thanks to my labmate Abhishek for his help during the clutter-robustness testing session described in Section 14.1.

References

[1] Cadene, R. et al. (2024). *LeRobot: State-of-the-art Machine Learning for Real-World Robotics.* Hugging Face.

[2] Zhao, T. Z., Kumar, V., Levine, S., and Finn, C. (2023). *Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware.* arXiv:2304.13705.

[3] Chi, C., Xu, Z., Feng, S., Cousineau, E., Du, Y., Burchfiel, B., Tedrake, R., and Song, S. (2024). *Diffusion Policy: Visuomotor Policy Learning via Action Diffusion.* arXiv:2303.04137.

[4] Hugging Face (2025). *SmolVLA: A Vision-Language-Action Model for Affordable and Efficient Robotics.*

[5] Wilson, E. B. (1927). Probable Inference, the Law of Succession, and Statistical Inference. *Journal of the American Statistical Association*, 22(158), 209-212. https://doi.org/10.1080/01621459.1927.10502953

[6] International Organization for Standardization. *ISO 18497:2018 - Agricultural machinery and tractors - Safety of highly automated agricultural machines.* (Referenced in the original project scope; not applicable to the delivered system - see Section 13.2.)

[7] International Electrotechnical Commission. *IEC 60204-1:2016 - Safety of machinery - Electrical equipment of machines.*

[8] International Electrotechnical Commission. *IEC 60825-1:2014 - Safety of laser products - Part 1: Equipment classification and requirements.* (Referenced in the original project scope; not applicable to the delivered system - see Section 13.2.)

Appendix A
Prepared Questions and Answers

This appendix reproduces the question-and-answer material prepared for the project's bench demonstration and interview. It is included here because the answers below restate several of this report's arguments in a more compressed, direct form, and may be useful as a starting point for a viva or defence, or as source material for an oral summary of the report's findings.

A.1 Why did you use a VLA / imitation learning instead of a classical inverse-kinematics approach?

My first hardware iteration (the Lynxmotion AL5D) used a classical pipeline: colour and AprilTag detection, a hand-calibrated pixel-to-world coordinate transform, and an inverse-kinematics solver. The coordinate-transform step was the main point of failure in practice. It needed careful calibration and broke under small changes in camera geometry or lighting. After that arm failed for an unrelated hardware reason, I moved to the LeRobot SO-101 and switched to imitation learning, where a policy learns the image-to-action mapping directly from demonstrations, meaning there was no explicit calibrated geometric model to build or maintain. With no calibration or geometry to keep up to date, the system tolerates small scene or camera changes that would silently break a hand-coded transform, exactly the failure mode I had already hit with the classical approach. The trade-off is that it needs many demonstrations to generalise, gives no formal accuracy guarantee, and is harder to debug: a failure like the bottom-tip grasp slip has to be diagnosed empirically instead of traced through a testable equation.

A.2 Why did you select a two-camera configuration (overhead and wrist)? Why not stereo?

I had an OAK-D Lite stereo/depth camera left over from the AL5D phase, so this was not a case of not knowing depth sensing was an option. The SO-101 lab rig came with its own two RGB webcams already mounted, wired, and working in LeRobot's camera pipeline. Bringing the OAK-D Lite across would have meant real integration work, a driver, remounting, recalibration, validating a depth channel through the whole pipeline, under a deadline already squeezed by the AL5D pivot. So I made a deliberate trade-off: use the already-working RGB cameras and spend the remaining time on demonstrations and training. The overhead camera gives global context, where the strawberry sits relative to the arm's base, while the wrist

camera gives a close, unoccluded view for the final approach, where the overhead view is foreshortened and blocked by the gripper. I confirmed this empirically when the wrist-camera mount physically shifted mid-project and every policy failed identically. With only the overhead camera I would expect that same failure by default; with only the wrist camera, coarse localisation of the fruit in the wider scene would be lost.

A.3 How would this system work in a more challenging environment (harsh lighting, clutter, and so on)? How can it be made more robust?

I would expect it to degrade, and I say that directly instead of overclaiming. Training and evaluation were both done under consistent lab lighting, on an uncluttered table with one prop. The policy sees RGB only, with no depth or lighting normalisation, and the demonstrations never showed distractor objects or shadow or glare conditions, so there is no evidence it has learned to be invariant to those conditions. I did not just assume this held; I tried to test it directly, and hit a concrete problem within the first attempt, which reinforces the concern rather than resolving it. Making it more robust would mean training data that deliberately varies lighting and background and not only the target's position; adding depth sensing so the policy is not purely appearance-based; and data augmentation or domain randomisation during training instead of assuming the deployment scene always matches the lab.

A.4 Did you try testing with clutter, for example placing the strawberry in a plastic plant?

Yes. After confirming backup motors were available, a labmate (Abhishek) and I dedicated a lab session to it. It did not go smoothly: the strawberry prop's weight meant it would not hold a stable position just sitting in the plant, so we tried wedging it between branches instead. That fixed the positioning but caused a new problem: the overhead camera could not see the strawberry at all once it was tucked among the branches. We tried a few other placements to keep it in view while still being "in clutter," and looked into whether there was a known way around this, but nothing resolved it in the time we had. So I could not get a valid test configuration running, let alone results, but the failure itself is informative. It is a direct, physical demonstration of the exact limitation described above: the system depends on an unoccluded RGB view and has no way to reason about a partially hidden target. That counts as a finding, not a gap in my testing.

A.5 Are there easy fixes for the failure cases?

Two low-effort options, and neither needs re-architecting. One is targeted re-recording of additional demonstrations specifically approaching from the bottom-tip angle, since the failure is concentrated in one specific approach direction, not spread across all grasp points. The other is a compliant, soft gripper fingertip, so a slightly mis-aligned grasp still holds instead of slipping. This pattern already has a track record: a gripper-release fault near light or dark contact areas was found and fixed earlier in the project by diagnosing the specific failure and then re-recording data for that exact case.

A.6 You report 80% (8/10) success: is that not too small a sample to state as a percentage?

That is exactly why I paired it with a 95% Wilson confidence interval: roughly 49-94% for 8/10. I am presenting it as a pilot or indicative result, not a validated benchmark; a meaningful benchmark would need on the order of 50-100 or more trials to narrow that interval.

A.7 Why only 10 evaluation trials?

The 10 trials are the final, formal evaluation batch, run after the last fine-tuning round: the clean, structured number I can back with a controlled before/after comparison. It is not the sum of all testing on the robot: throughout the project I ran rollouts after every training or fine-tuning round, watching the arm's behaviour live and flagging anything that looked like an outlier. That informal testing is how the wrist-camera-mount shift and the gripper-release fault were caught; neither showed up in a log, only by watching the robot. The final 10-trial run is the number I am willing to formally quote; it sits on top of a much larger amount of qualitative testing. Given more lab time, I would run a much larger structured batch instead.

A.8 What exactly counted as a "success" versus a "failure" in your trials?

A success was a clean pick: gripper closes on the strawberry prop without crushing or dropping it, lifts it, and places it in the target location. Both failures were the same specific mode: grasping near the bottom tip in preference to the stem/hook or middle body, causing a slip.

A.9 Why did you move from the AL5D to the LeRobot SO-101 specifically?

The AL5D had a manufacturing defect stalling hardware progress with no quick fix. My supervisor offered access to the lab's SO-101 rig, already set up with dual cameras and ready to use, letting me keep the project moving without a long hardware-sourcing delay.

A.10 Why SmolVLA specifically, rather than another pretrained VLA?

It is small enough to fine-tune and run on the hardware I had (Jetson AGX Orin for training, SO-101 for inference), it is directly integrated into the LeRobot framework I was already using for ACT and Diffusion Policy, and it is actively maintained with a clear fine-tuning workflow, which mattered given limited project time.

A.11 What would you do differently if you started again?

Run a much larger, more varied evaluation set before quoting any success-rate number, and build lighting and clutter variation into the demonstration data from the start instead of leaving it as future work; both directly address the two weakest points in the current results.

APPENDIX B
REPRODUCIBILITY REFERENCE

This appendix records the exact commands and configuration used throughout the project, for reproducibility and for anyone continuing this work. All commands assume LeRobot version 0.5.2 (Section 5.1) with the appropriate environment activated: `conda activate lerobot` on the recording and evaluation desktop, or the Jetson's dedicated Python virtual environment for training.

B.1 Calibration

```
lerobot-calibrate --robot.type=so101_follower \
    --robot.port=/dev/ttyACM1 \
    --robot.id=so101_follower_arm
lerobot-calibrate --teleop.type=so101_leader \
    --teleop.port=/dev/ttyACM0 \
    --teleop.id=so101_leader_arm
```

B.2 Camera Configuration (used throughout recording)

```
--robot.cameras="{
  top: {type: opencv, index_or_path: /dev/video0,
        width: 640, height: 480, fps: 30},
  wrist: {type: opencv, index_or_path: /dev/video2,
          width: 640, height: 480,
          fps: 30, rotation: 180}
}"
```

B.3 Training (SmolVLA fine-tune, the project's main policy)

```
lerobot-train \
    --policy.path=lerobot/smolvla_base \
    --dataset.repo_id=<repo> \
    --output_dir=outputs/train/<name> \
    --job_name=<name> \
    --policy.device=cuda --steps=20000 \
    --batch_size=8 \
    --policy.push_to_hub=true \
    --policy.repo_id=<hub_repo>
```

Run on the Jetson AGX Orin (Section 4.4), not the recording/evaluation desktop.

B.4 Real-Robot Evaluation

`lerobot-rollout` was used for evaluation, not `lerobot-record`: in LeRobot 0.5.2, `lerobot-record` is teleoperation-only and has no `--policy.path` argument.

```
lerobot-rollout \
    --strategy.type=episodic \
    --policy.path=<local snapshot dir - see Sec 12.3> \
    --robot.type=so101_follower \
    --robot.port=/dev/ttyACM1 \
    --robot.id=so101_follower_arm \
    --robot.cameras='{...same block as B.2...}' \
    --task="<task string>" --duration=60
```

B.5 Asynchronous Inference (required for SmolVLA)

Synchronous inference via `lerobot-rollout` alone was measured at only 4.3-4.7 Hz for SmolVLA, unusable for live control (Section 4.4). The asynchronous server/client workflow was used instead for every SmolVLA evaluation:

```
cd eval_results
./run_eval.sh server      # terminal 1, left running
./run_eval.sh strawberry  # terminal 2
  # options: demo | strawberry-cube | blacktip
```

This uses a `chunk_size_threshold` of 0.3. The policy being evaluated is negotiated at client handshake time, so switching which model is under test does not require restarting the server.

B.6 Known Configuration Gotchas

`pretrained_revision` **field mismatch** (Section 12.3): if loading a model straight from its Hugging Face Hub repository fails with `draccus.utils.DecodingError: The fields 'pretrained_revision' are not valid for <PolicyName>Config`, download the model locally with `snapshot_download`, strip the `pretrained_revision` field from the downloaded `config.json`, and point `--policy.path` at the local directory instead of the Hub repository ID.

Hugging Face authentication check is unreliable. `hf auth login` without the `--force` flag only checks that a token file exists locally, not that it is still valid server-side. If a command fails with an "Invalid user token" error despite a token file being present, run `hf auth login --force` and provide a fresh token with write scope.

B.7 Hugging Face Hub Model Repositories

All under the `Akshit03/` namespace:

B.8 Source Code

The project's aggregation scripts, training and evaluation launch scripts, demonstration-recording scripts, and camera-diagnostic tooling are published at https://github.com/akshitharsola/learning-to-pick-lerobot-so101. Recorded datasets and trained model checkpoints are not included there; both are hosted on the Hugging Face Hub under the `Akshit03/` namespace, as listed in Table VII and Section 8.4.

TABLE VII
HUGGING FACE HUB MODEL REPOSITORIES

Model	Task(s)	Notes
act_mir1_v2	cube	v2-round ACT, trained from scratch
diffusion_mir1_v2	cube	v2-round Diffusion Policy, trained from scratch
smolvla_mir1_v2	cube	v2-round SmolVLA baseline, fine-tuned from lerobot/smolvla_base
smolvla_mir1_blacktip_fix	cube	Round 3 - gripper-release fault fixed
smolvla_mir1_strawberry	cube + strawberry	Round 4 - final policy, evaluated in Chapter 10